



Eötvös Loránd University
Faculty of Informatics

Introduction to CSS3

Web-development I. course

What is CSS?

- CSS = **Cascading Style Sheets**
- W3C standard for describing the presentation of HTML elements.
- CSS is a language in that it has its own syntax rules.



What does cascading mean?

- Cascading: The process of combining several style sheets and resolving conflicts between them.

Håkon Wium Lie (CSS co-creator)



Benefits of CSS

- **Improved control over formatting.** The degree of formatting control in CSS is significantly better than that provided in HTML.
- **Improved site maintainability.** Web sites become significantly more maintainable because all formatting can be centralized into one, or a small handful, of CSS files.
- **Improved accessibility.** CSS-driven sites are more accessible.
- **Improved page download speed.** A site built using a centralized set of CSS files for all presentation will also be quicker to download because each individual HTML file will contain less markup.
- **Improved output flexibility.** CSS can be used to adopt a page for different output mediums.

Same content, different design

- <http://www.csszengarden.com/>



Mid Century Modern
ANDREW LOHMAN, United States



Garments
DAN MALL, United States



Steel
STEFFEN KNOELLER, Germany



Apothecary
TRENT WALTON, United States



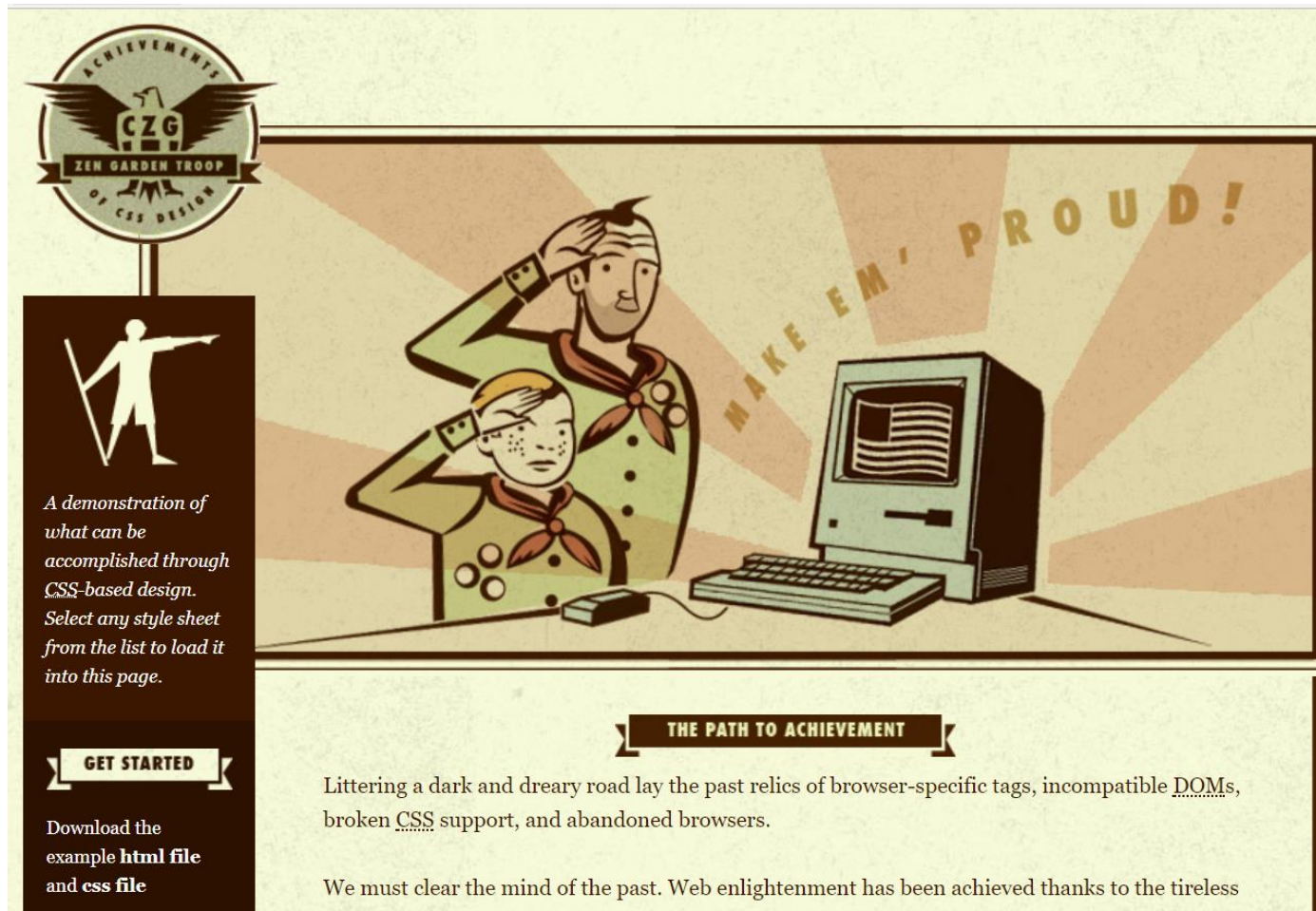
Handcrafted
ELLIOT JAY STOCKS, United Kingdom



Fountain Kiss
JEREMY CARLSON, United States

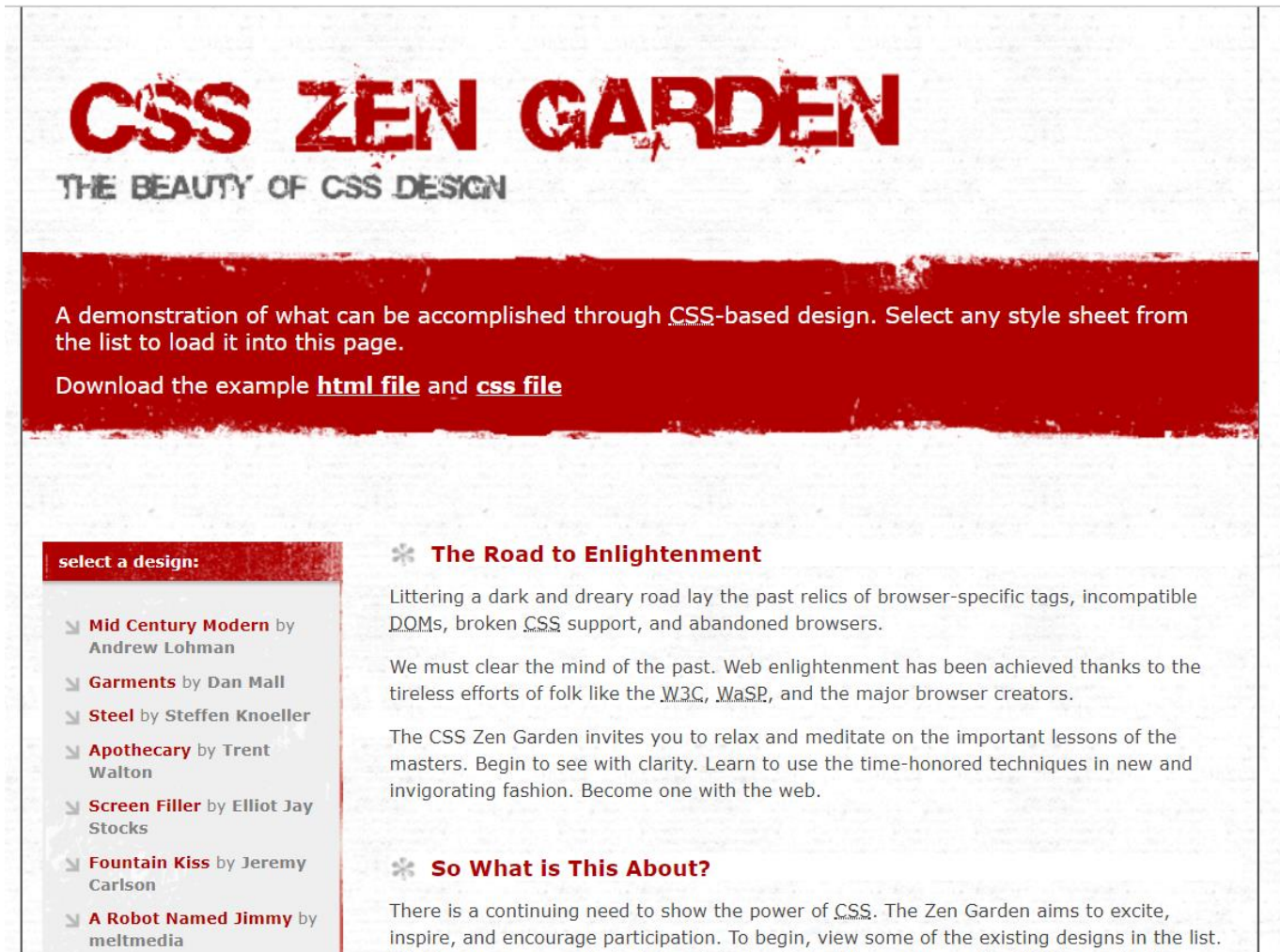
Same content, different design

- <http://www.csszengarden.com/>



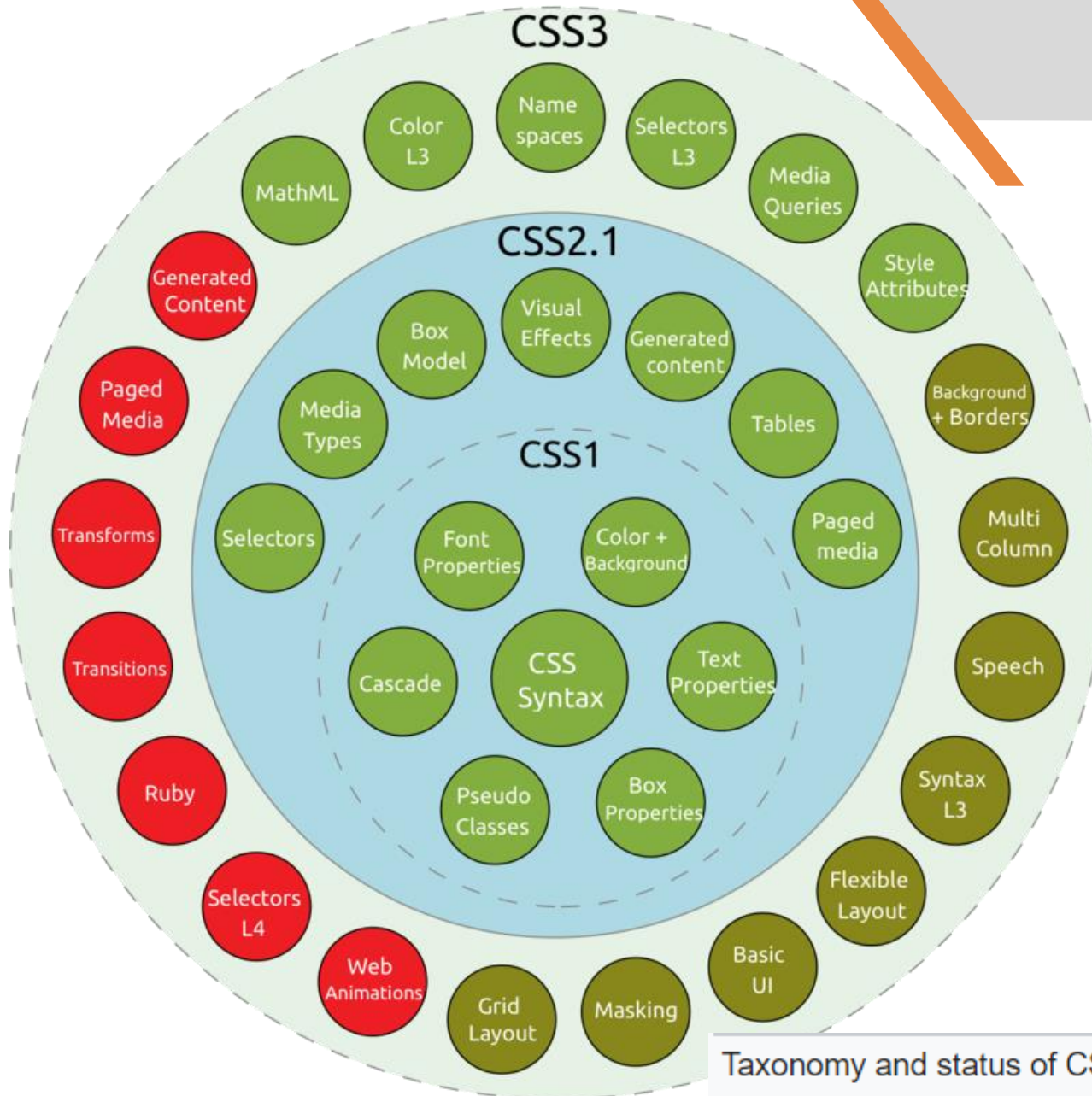
Same content, different design

- <http://www.csszengarden.com/>





History of CSS



Taxonomy and status of CSS3 **modules**.

- Recommendation
- Candidate Recommendation
- Last Call
- Working Draft.

CSS current work

<https://www.w3.org/Style/CSS/current-work>

Stable	Current	Up-coming	Abbreviation	Full name
CSS Color Level 3	CR	PR	FPWD	First Public Working Draft
Selectors Level 3	CR	PR	WD	Working Draft
CSS Backgrounds and Borders Level 3	CR	PR	CR	Candidate Recommendation
			PR	Proposed Recommendation
CSS Conditional Rules Level 3	CR	CR	REC	Recommendation
CSS Multi-column Layout Level 1	WD	CR		
CSS Values and Units Level 3	CR	PR		
CSS Cascading and Inheritance Level 3	CR	PR		



CSS syntax

CSS rules

A CSS rule contains two parts: a **selector** and a **declaration**.

SELECTOR



p {

font-family: Arial; }



DECLARATION

Selectors indicate which element the rule applies to.

This rule indicates that all **<p>** elements should be shown in the Arial typeface.

CSS rules

Declarations indicate how the elements referred to in the selector should be styled.

SELECTOR



p {

font-family: Arial;}



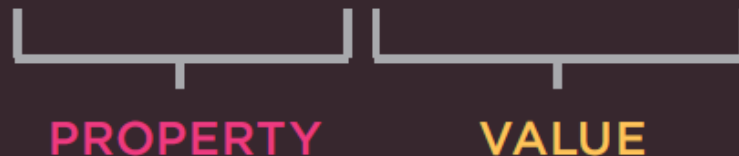
DECLARATION

Declarations are split into two parts (a property and a value), and are separated by a colon.

CSS declaration

You can specify several properties in one declaration, each separated by a semi-colon.

```
h1 {  
    font-family: Arial;  
    color: yellow;}  
    
```



CSS declaration

The same rule can apply to more than one element if you separate the element names with commas.

```
h1, h2, h3 {  
  font-family: Arial;  
  color: yellow;}  
  └──┬──┘ └──┬──┘  
      PROPERTY  VALUE
```

Selector chain



This rule indicates that all **<h1>**, **<h2>** and **<h3>** elements should be shown in the Arial typeface, in a yellow color.

CSS declaration

Properties indicate the aspects of the element you want to change. For example, color, font, width, height and border.

```
h1, h2, h3 {  
    font-family: Arial;  
    color: yellow;}  
└──────────┘ └──────────┘  
  PROPERTY    VALUE
```

Each CSS declaration also contains a **value** for a property.

- The unit of any given value is dependent upon the property.
- Some property values are from a predefined list of keywords.
- Others are values such as length measurements, percentages, numbers without units, color values, and URLs.

Unknown properties

User agents must ignore a declaration with an unknown property/illegal value.

<code>img { float: left }</code>	<code>/* correct CSS declaration*/</code>
<code>img { float: left here }</code>	<code>/* „here” is not a valid value of float */</code>
<code>img { background: "red" }</code>	<code>/* keywords cannot be quoted*/</code>
<code>img { border-width: 3 }</code>	<code>/* a unit must be specified*/</code>

A CSS parser would honor the first rule and ignore the rest, as if the style sheet had been:

```
img { float: left }  
img {}  
img {}  
img {}
```

Properties in action

<http://htmlcheatsheet.com/css/>

Grumpy wizards make toxic
brew for the evil Queen and
Jack.

Font:

Size:

Letter spc:

Word spc:

Color:

Weight:

Decoration:

Style:

Variant:

Case:

```
font-family: Georgia, serif;
font-size: 21px;
letter-spacing: 2.6px;
word-spacing: 3.6px;
color: #000000;
font-weight: normal;
text-decoration: none;
```

*GRUMPY WIZARDS MAKE TOXIC BREW FOR
THE EVIL QUEEN AND JACK*

Font:

Size:

Letter spc:

Word spc:

Color:

Weight:

Decoration:

Style:

Variant:

Case:

```
font-family: "Times New Roman", Times, serif;
font-size: 21px;
letter-spacing: -1.6px;
word-spacing: 3.2px;
color: #FF2E74;
font-weight: 700;
text-decoration: none solid rgb(68, 68, 68);
```

Comments in CSS

- It is often helpful to add comments to your style sheets.
- Comments take the form: */* comment goes here */*

```
62 footer p {text-align:center}
63
64 article:last-of-type {border:0;}
65
66 /* end of page structure */
67
68 /* navigation */
69 header nav {
70     background-color:#ddd;
71     padding:5px 5px 5px 10px;
72     margin-left:-10px;
73     margin-right:-10px;
74     border-top:1px solid black;
75     border-bottom:1px solid black;
76 }
77
```


Validating stylesheets

- <http://jigsaw.w3.org/css-validator/>

Deutsch English Español Français 한국어 Italiano Nederlands 日本語 Polski Portu



CSS Validation Service

Check Cascading Style Sheets (CSS) and (X)HTML documents with style sheets

By URI

By file upload

By direct input

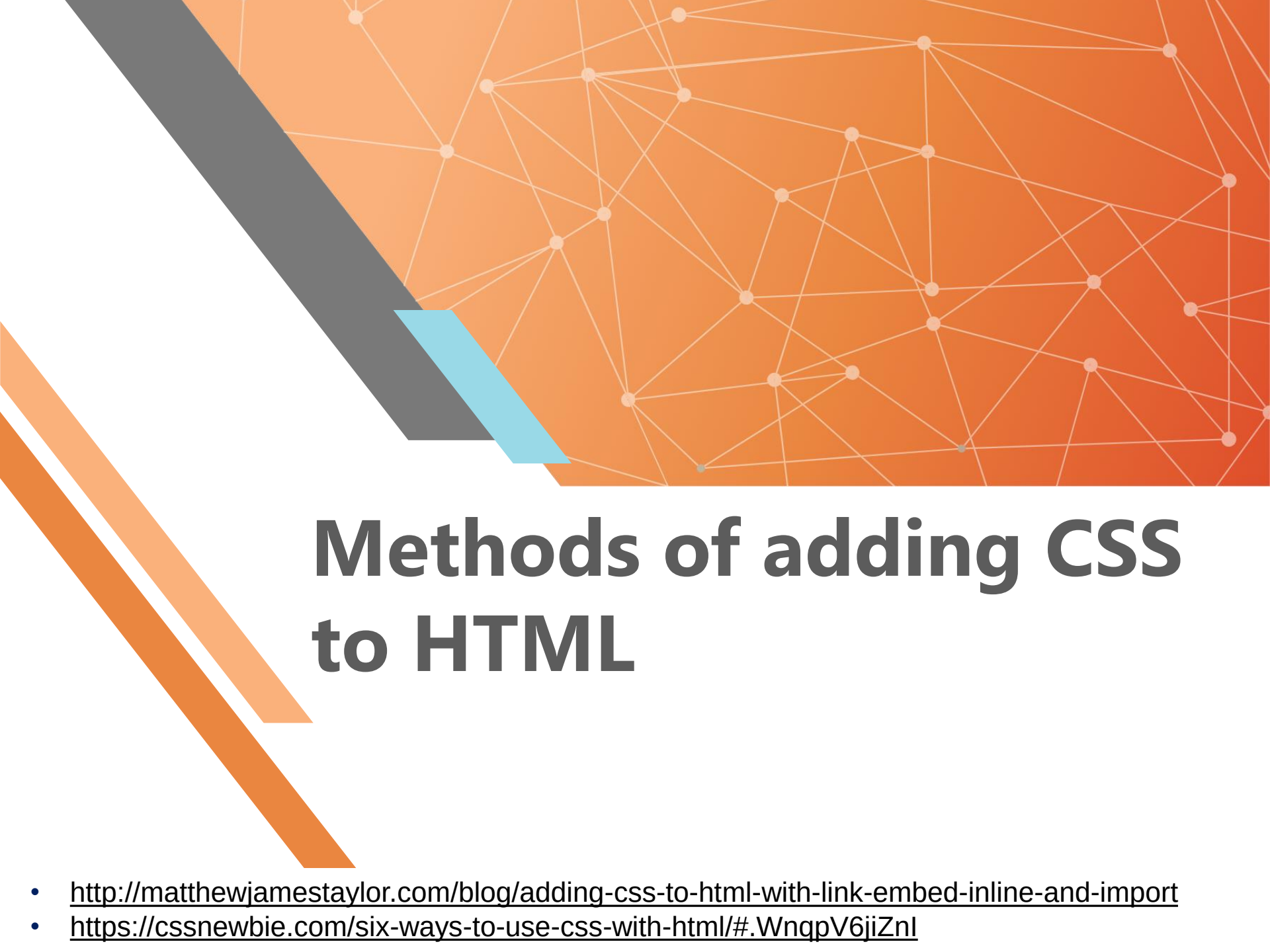
Validate by URI

Enter the URI of a document (HTML with CSS or CSS only) you would like validated:

Address:

► More Options

Check



Methods of adding CSS to HTML

- <http://matthewjamestaylor.com/blog/adding-css-to-html-with-link-embed-inline-and-import>
- <https://cssnewbie.com/six-ways-to-use-css-with-html/#.WnqpV6jiZnI>

Different type of style sheets

- **Author-created style sheets** (what we are learning in this Course).
- **User style sheets** allow the individual user to tell the browser to display pages using that individual's own custom style sheet. This option is available in a browser usually in its accessibility options area.
- The **browser style sheet** defines the default styles the browser uses for each HTML element.

Locations of author created styles

Linking to a separate CSS file (external css)

Embedding CSS into the HTML (embedded/internal stylesheet)

Importing a CSS file from within CSS

Adding Inline CSS to HTML tags

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <title>CSS example</title>
    <link rel="stylesheet" type="text/css"
      href="css/style.css" media="screen">
    <style type="text/css">
      @import url("css/highcontrast.css");
      h1 {color: blue}
    </style>
  </head>

  <body>
    <h1>Heading 1</h1>
    <h2>Heading 2</h2>
    <p style="color:green">This is a green
      paragraph</p>
  </body>
</html>
```

Adding Inline CSS to HTML tags

Style rules can also be added directly to any HTML element. To do this you simply add a style parameter to the element and enter your style rules as the value.

```
<p style="color:green">  
  This is a green paragraph  
</p>
```

This is not a good method to use because it will bloat your HTML and make website maintenance a real headache.

However it can be useful in some situations, e.g. if you are using a system where you don't have access to the CSS file - simply add your style rules directly to the elements instead.

Inline example

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Inline example</title>
</head>
<body>
<h1 style="color:blue">Heading 1</h1>
<p>Lorem ipsum dolor sit amet, consectetur adipiscing
elit.</p>
<h1 style="color:blue">Heading 1 </h1>
<p>Lorem ipsum dolor sit amet, consectetur adipiscing
elit.</p>
<h1 style="color:blue">Heading 1 </h1>
<p>Lorem ipsum dolor sit amet, consectetur adipiscing
elit.</p>
</body>
</html>
```



Embedding CSS into the HTML (internal CSS)

You can embed CSS rules directly into any HTML page. To do this you need to add the following code to the <head> of your HTML document:

```
<style type="text/css">
```

```
</style>
```

Example

```
<style type="text/css">
```

```
  h1 {color:green}
```

```
</style>
```


Embedding CSS into the HTML (internal CSS)

The disadvantage: the styles must be downloaded every time someone visits the page (slower browsing experience)

However there are a few advantages.

Because the CSS is part of the HTML document, the whole page exists as just one file. This can be useful if you are sending the page to someone via email or if it will be used as a template such as a blogger template.

Inline CSS ► internal CSS

```
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<title>Inline to internal</title>
<style type="text/css">
  h1 {color:blue}
</style>
</head>
<body>
```

```
<h1>Heading 1 </h1>
<p>Lorem ipsum dolor sit amet, consectetur adipiscing elit.</p>
<h1>Heading 1 </h1>
<p>Lorem ipsum dolor sit amet, consectetur adipiscing elit.</p>
<h1>Heading 1 </h1>
<p>Lorem ipsum dolor sit amet, consectetur adipiscing elit.</p>
</body>
</html>
```

Using link tag (external CSS)

- There are many advantages to linking to a separate CSS file.
- If you need to make a style change across your whole website then you only need to make the change once in your single CSS file.
- If you want to completely change the look of your website, again, you only need to update this one file.

Using link tag (external CSS)

```
<head>
```

```
  <link rel="stylesheet" type="text/css"  
  href="css/style.css">
```

```
</head>
```

<link>: Empty element (it does not need a closing tag)

- **href**: This specifies the path to the CSS file (which is often placed in a folder called css or styles).
- **type**: This attribute specifies the type of document being linked to. The value should be text/css.
- **rel**: This specifies the relationship between the HTML page and the file it is linked to. The value should be stylesheet when linking to a CSS file.

Same example with external CSS

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>external css</title>
  <link rel="stylesheet" type="text/css" href="style.css">
</head>
<body>
<h1>Heading 1</h1>
<p>Lorem ipsum dolor sit amet, consectetur adipiscing elit.</p>
<h1>Heading 1 </h1>
<p>Lorem ipsum dolor sit amet, consectetur adipiscing elit.</p>
<h1>Heading 1 </h1>
<p>Lorem ipsum dolor sit amet, consectetur adipiscing elit.</p>
</body>
</html>
```

The content of the style.css file is:
h1 {color:blue}

Importing a CSS file from within CSS

- Another way to add CSS to a HTML page is with the import rule. This lets us attach a new CSS file from within CSS itself.

```
@import  
url ("css/highcontrast.css") ;
```

Combining Internal, External, & Inline

Combining these practices of CSS styling is an easy way to customize your page exactly how you want it.

You can have a site wide stylesheet, page specific styling, and a few element specific style adjustments to make your page exactly how you want it.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <title>CSS example</title>
    <link rel="stylesheet" type="text/css"
      href="css/style.css" media="screen">
    <style type="text/css">
      @import url("css/highcontrast.css");
      h1 {color: blue}
    </style>
  </head>

  <body>
    <h1>Heading 1</h1>
    <h2>Heading 2</h2>
    <p style="color:green">This is a green
      paragraph</p>
  </body>
</html>
```

Alter elements through Javascript

- Using javascript you can alter elements styling and adjust them as needed.
- Usually you would need some sort of user interaction or change that would take place and you would alter an element on your page.
- A common use for this would be a “show more” button for a descript that unhides more text for the user.

```
<div id="show_more" style="display: none;">  
    Additional text!  
</div>
```

```
<script>  
document.getElementById('show_more').style.display = 'block';  
</script>
```




Units

Units of Measurement

- Two type of units:
 - **relative units:** they are based on the value of something else, such as the size of a parent element.
 - **absolute units:** they have a real-world size.

Unless you are defining a style sheet for printing, it is recommended to avoid using absolute units.

Pixels are perhaps the one popular exception (but there are also good reasons for avoiding the pixel unit in some cases).

Relative Units

Unit	Description	Type
px	Pixel. In CSS2 this is a relative measure, while in CSS3 it is absolute (1/96 of an inch).	Relative (CSS2) Absolute (CSS3)
em	Equal to the computed value of the font-size property of the element on which it is used. When used for font sizes, the em unit is in relation to the font size of the parent.	Relative
%	A measure that is always relative to another value. The precise meaning of % varies depending upon which property it is being used.	Relative
ex	A rarely used relative measure that expresses size in relation to the x-height of an element's font.	Relative

Relative Units

Unit	Description	Type
ch	Another rarely used relative measure; this one expresses size in relation to the width of the zero ("0") character of an element's font.	Relative (CSS3 only)
rem	Stands for root em, which is the font size of the root element. Unlike em , which may be different for each element, the rem is constant throughout the document.	Relative (CSS3 only)
vw, vh	Stands for viewport width and viewport height. Both are percentage values (between 0 and 100) of the viewport (browser window). This allows an item to change size when the viewport is resized.	Relative (CSS3 only)

Absolute Units

Unit	Description	Type
in	Inches	Absolute
cm	Centimeters	Absolute
mm	Millimeters	Absolute
pt	Points (equal to 1/72 of an inch)	Absolute
pc	Pica (equal to 1/6 of an inch)	Absolute

The rem unit

- CSS3 now supports a new relative measure, the **rem** (for root em unit).
- This unit is always relative to the size of the root element (i.e., the `<html>` element).
- However, since Internet Explorer prior to version 9 do not support the rem units, you need to provide some type of fallback for those browsers.

Recommended video

CSS Units (CSS Lengths: rems, ems, pixels, percents, and more)

<https://www.youtube.com/watch?v=qrdUUDxBSY>



URL

- The `<url>` data type is specified using the `url()` functional notation.
- It may be written without quotes, or surrounded by single or double quotes. Relative URLs are allowed, and are relative to the URL of the stylesheet
- **`url("url");`**
 - *Example*
 - `body { background-image: url('images/bg.jpg'); }`
 - `li { list-style: url('http://somewhere.org/flower.png') disc }`

The background features a complex geometric design. In the top right, there is a large orange area with a network of white lines connecting small dots. On the left side, there are several diagonal stripes: a thick grey one, a thinner light blue one, and two orange ones of varying shades. The title 'Colors and opacity' is centered in the lower half of the image.

Colors and opacity

Beginning Graphic Design: Color

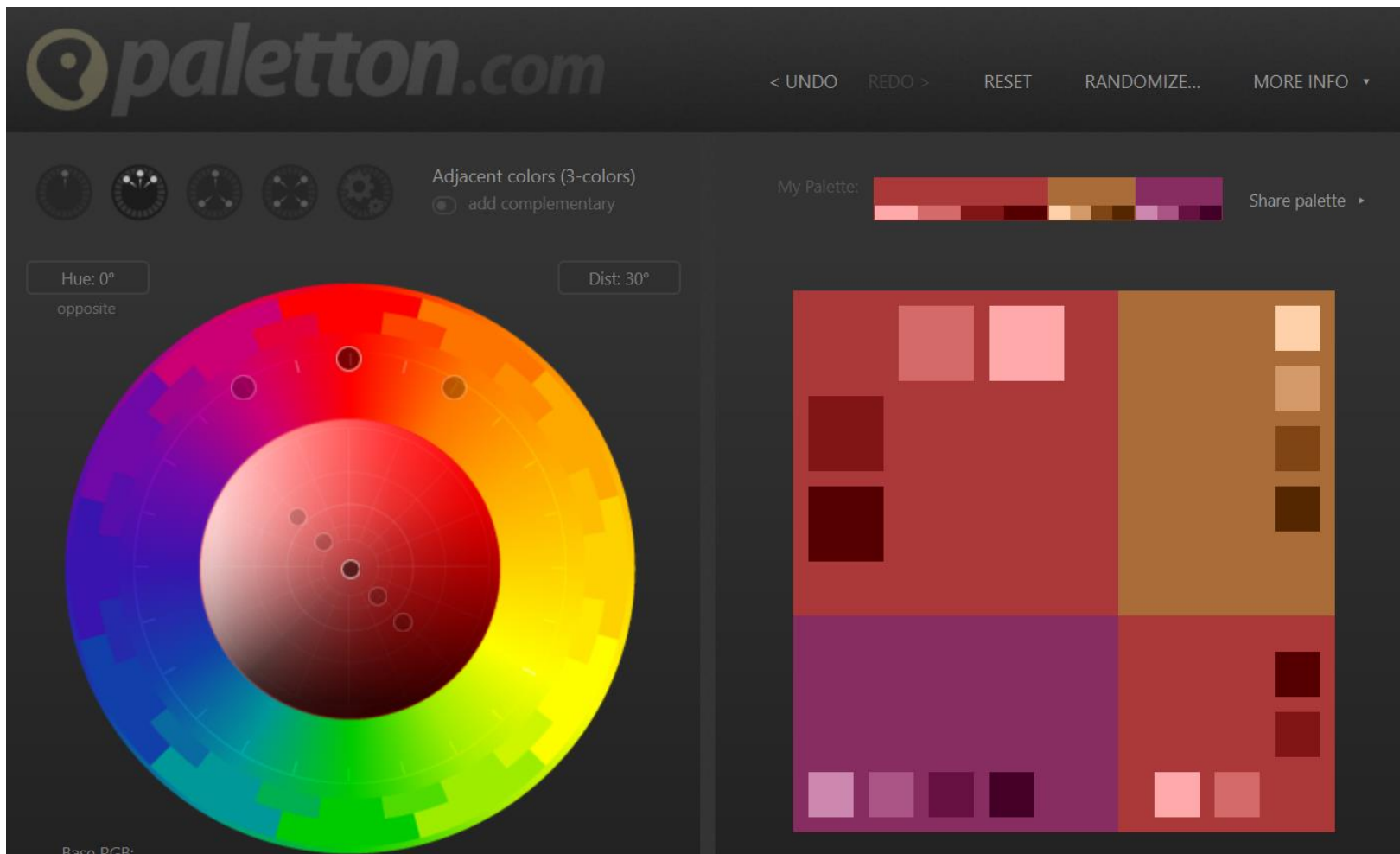


https://www.youtube.com/watch?v=_2LLXnUdUlc



Choosing colors...

- <http://paletton.com/>



Understanding color

HUE

Hue is near to the colloquial idea of color. Technically speaking however, a color can also have saturation and brightness as well as hue.



SATURATION

Saturation refers to the amount of gray in a color. At maximum saturation, there would be no gray in the color. At minimum saturation, the color would be mostly gray.



BRIGHTNESS

Brightness (or "value") refers to how much black is in a color. At maximum brightness, there would be no black in the color. At minimum brightness, the color would be very dark.



Color values (hex, rgba, and color names)

webflow

<https://youtu.be/JBRZ3KBfOk8>



Understanding color

RGB VALUES

Values for red, green, and blue are expressed as numbers between 0 and 255.



`rgb(102,205,170)`

This color is made up of the following values:

102 red
205 green
170 blue

HEX CODES

Hex values represent values for red, green, and blue in hexadecimal code.



`#66cdaa`

The value of the red, 102, is expressed as 66 in hexadecimal code. The 205 of the green is expressed as cd and the 170 of the blue equates to aa.

COLOR NAMES

Colors are represented by predefined names. However, they are very limited in number.



`MediumAquaMarine`

There are 147 color names supported by browsers (this color is `MediumAquaMarine`). Most consider this to be a limited color palette, and it is hard to remember the name for each of the colors so (apart from white and black) they are not commonly used.

Color Values

- CSS supports a variety of different ways of describing color

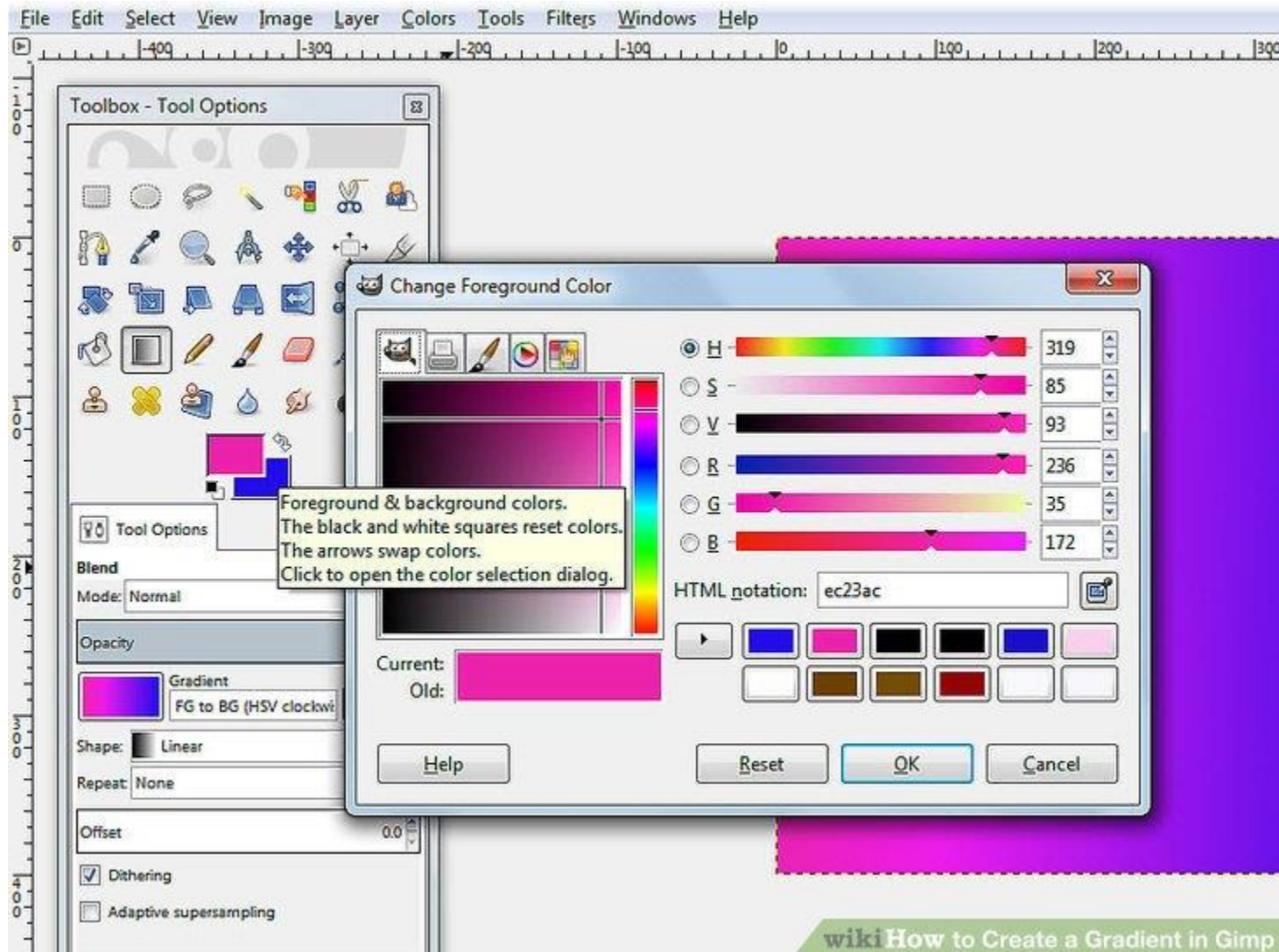
Method	Description	Example
Name	Use one of 17 standard color names. CSS3 has 140 standard names.	color: red; color: hotpink; /* CSS3 only */
RGB	Uses three different numbers between 0 and 255 to describe the Red, Green, and Blue values for the color.	color: rgb(255,0,0); color: rgb(255,105,180);
Hexadecimal	Uses a six-digit hexadecimal number to describe the red, green, and blue value of the color; each of the three RGB values is between 0 and FF (which is 255 in decimal). Notice that the hexadecimal number is preceded by a hash or pound symbol (#).	color: #FF0000; color: #FF69B4;

Color Values

- CSS supports a variety of different ways of describing color

Method	Description	Example
RGBA	Allows you to add an alpha, or transparency, value. This allows a background color or image to “show through” the color. Transparency is a value between 0.0 (fully transparent) and 1.0 (fully opaque).	<code>color: rgb(255,0,0, 0.5);</code>
HSL	Allows you to specify a color using Hue Saturation and Light values. This is available only in CSS3. HSLA is also available as well.	<code>color: hsl(0,100%,100%);</code> <code>color: hsl(330,59%,100%);</code>

Colors in GIMP software



Color examples

```
/* color name */
h1 {
  color: DarkCyan;}
/* hex code */
h2 {
  color: #ee3e80;}
/* rgb value */
p {
  color: rgb(100,100,90);}
```

The **color** property allows you to specify the color of text inside an element.

See
<https://rgbcolorcode.com/>

RESULT

Marine Biology

The Composition of Seawater

Almost anything can be found in seawater. This includes dissolved materials from Earth's crust as well as materials released from organisms. The most important components of seawater that influence life forms are salinity, temperature, dissolved gases (mostly oxygen and carbon dioxide), nutrients, and pH. These elements vary in their composition as well as in their influence on marine life.

Color examples

```
body {  
  background-color: rgb(200,200,200);}  
h1 {  
  background-color: DarkCyan;}  
h2 {  
  background-color: #ee3e80;}  
p {  
  background-color: white;}
```

RESULT

Marine Biology

The Composition of Seawater

Almost anything can be found in seawater. This includes dissolved materials from Earth's crust as well as materials released from organisms. The most important components of seawater that influence life forms are salinity, temperature, dissolved gases (mostly oxygen and carbon dioxide), nutrients, and pH. These elements vary in their composition as well as in their influence on marine life.

You can specify your choice of background color in the same ways you can specify foreground colors

If you do not specify a background color, then the background is transparent.

Color examples

CSS

chapter-11/hsla.html

```
body {
  background-color: #C8C8C8;
  background-color: hsl(0,0%,78%);}
p {
  background-color: #ffffff;
  background-color: hsla(0,100%,100%,0.5);}
```

RESULT

Marine Biology

The Composition of Seawater

Almost anything can be found in seawater. This includes dissolved materials from Earth's crust as well as materials released from organisms. The most important components of seawater that influence life forms are salinity, temperature, dissolved gases (mostly oxygen and carbon dioxide), nutrients, and pH. These elements vary in their composition as well as in their influence on marine life.

The hsl color property has been introduced in CSS3 as an alternative way to specify colors.

hue

This is expressed as an angle (between 0 and 360 degrees).

saturation

This is expressed as a percentage.

lightness

This is expressed as a percentage with 0% being white, 50% being normal, and 100% being black.

alpha

This is expressed as a number between 0 and 1.0.

Contrast

- When picking foreground and background colors, it is important to ensure that there is enough contrast for the text to be legible.



LOW
CONTRAST

Text is harder to read when there is low contrast between background and foreground colors.



HIGH
CONTRAST

Text is easier to read when there is higher contrast between background and foreground colors.



MEDIUM
CONTRAST

For long spans of text, reducing the contrast a little bit improves readability.

Opacity

The value is a number between **0.0** and **1.0** (so a value of **0.5** is 50% opacity and **0.15** is 15% opacity).

CSS

chapter-11/opacity.html

```
p.one {  
  background-color: rgb(0,0,0);  
  opacity: 0.5;}  
p.two {  
  background-color: rgb(0,0,0);  
  background-color: rgba(0,0,0,0.5);}
```

RESULT

CSS3 introduces the **opacity** property which allows you to specify the opacity of an element and any of its child elements.

Opacity (background)

The CSS3 **rgba** property allows you to specify a color, but adds a fourth value to indicate opacity.

This value is known as an **alpha** value.

CSS

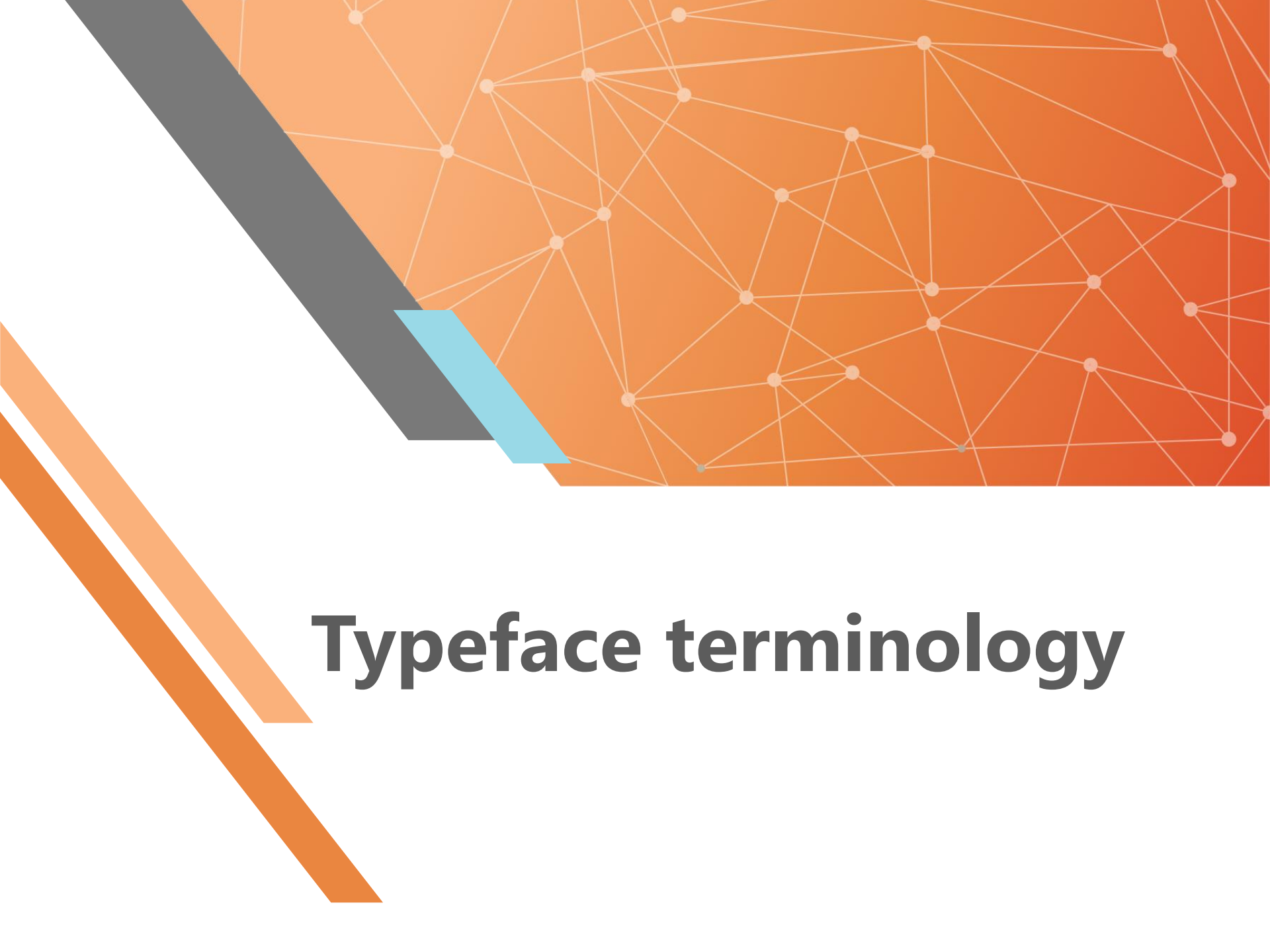
chapter-11/opacity.html

```
p.one {  
  background-color: rgb(0,0,0);  
  opacity: 0.5;}  
p.two {  
  background-color: rgb(0,0,0);  
  background-color: rgba(0,0,0,0.5);}
```

RESULT



The **rgba** value will only affect the element on which it is applied (not child elements).



Typeface terminology

Beginning Graphic Design: Typography



TYPOGRAPHY

<https://www.youtube.com/watch?v=sByzHoiYFX0>



Typeface

SERIF

Serif fonts have extra details on the ends of the main strokes of the letters. These details are known as serifs.



In print, serif fonts were traditionally used for long passages of text because they were considered easier to read.

SANS-SERIF

Sans-serif fonts have straight ends to letters, and therefore have a much cleaner design.



Screens have a lower resolution than print. So, if the text is small, sans-serif fonts can be clearer to read.

MONOSPACE

Every letter in a monospace (or fixed-width) font is the same width. (Non-monospace fonts have different widths.)



Monospace fonts are commonly used for code because they align nicely, making the text easier to follow.

Typeface

SERIF

Serif fonts have extra details on the end of the main strokes of the letters.

EXAMPLES:

Georgia

Times

Times New Roman

SANS-SERIF

Sans-serif fonts have straight ends to letters and therefore have a much cleaner design.

EXAMPLES:

Arial

Verdana

Helvetica

As a result, sites often use a small set of typefaces that are installed on most computers (shown above). There are some techniques to get around this limitation...

Typeface

MONOSPACE

Every letter in a monospace typeface is the same width. (Non-monospace fonts have different widths.)

EXAMPLES:

Courier

Courier New

CURSIVE

Cursive fonts either have joining strokes or other cursive characteristics, such as handwriting styles.

EXAMPLES:

Comic Sans MS

Monotype Corsiva

FANTASY

Fantasy fonts are usually decorative fonts and are often used for titles. They're not designed for long bodies of text.

EXAMPLES:

Impact

Haettenschweiler

Typeface

WEIGHT

Light

Medium

Bold

Black

The font weight not only adds emphasis but can also affect the amount of white space and contrast on a page.

STYLE

Normal

Italic

Oblique

Italic fonts have a cursive aspect to some of the lettering. Oblique font styles take the normal style and put it on an angle.

STRETCH

Condensed

Regular

Extended

In condensed (or narrow) versions of the font, letters are thinner and closer together. In expanded versions they are thicker and further apart.

Text properties

CSS provides two types of properties that affect text.

- **font properties** that affect the font and its appearance.
- **text properties** that affect the text in a similar way no matter which font is being used.

Font properties

Property	Description
<code>font</code>	A combined shorthand property that allows you to set the family, style, size, variant, and weight in one property. While you do not have to specify each property, you must include at a minimum the font size and font family. In addition, the order is important and must be: <code>style weight variant size font-family</code>
<code>font-family</code>	Specifies the typeface/font (or generic font family) to use. More than one can be specified.
<code>font-size</code>	The size of the font in one of the measurement units.
<code>font-style</code>	Specifies whether <code>italic</code> , <code>oblique</code> (i.e., skewed by the browser rather than a true italic), or <code>normal</code> .
<code>font-variant</code>	Specifies either <code>small-caps</code> text or <code>none</code> (i.e., regular text).
<code>font-weight</code>	Specifies either <code>normal</code> , <code>bold</code> , <code>bolder</code> , <code>lighter</code> , or a value between 100 and 900 in multiples of 100, where larger number represents weightier (i.e., bolder) text.

Text properties

Property	Description
<code>letter-spacing</code>	Adjusts the space between letters. Can be the value <code>normal</code> or a length unit.
<code>line-height</code>	Specifies the space between baselines (equivalent to leading in a desktop publishing program). The default value is <code>normal</code> , but can be set to any length unit. Can also be set via the shorthand <code>font</code> property.
<code>list-style-image</code>	Specifies the URL of an image to use as the marker for unordered lists.
<code>list-style-type</code>	Selects the marker type to use for ordered and unordered lists. Often set to <code>none</code> to remove markers when the list is a navigational menu or a input form.
<code>text-align</code>	Aligns the text horizontally in a container element in a similar way as a word processor. Possible values are <code>left</code> , <code>right</code> , <code>center</code> , and <code>justify</code> .
<code>text-decoration</code>	Specifies whether the text will have lines below, through, or over it. Possible values are: <code>none</code> , <code>underline</code> , <code>overline</code> , <code>line-through</code> , and <code>blink</code> . Hyperlinks by default have this property set to <code>underline</code> .

Text properties

<code>text-direction</code>	Specifies the direction of the text, left-to-right (<code>ltr</code>) or right-to-left (<code>rtl</code>).
<code>text-indent</code>	Indents the first line of a paragraph by a specific amount.
<code>text-shadow</code>	A new CSS3 property that can be used to add a drop shadow to a text. Not yet supported in IE9.
<code>text-transform</code>	Changes the capitalization of text. Possible values are <code>none</code> , <code>capitalize</code> , <code>lowercase</code> , and <code>uppercase</code> .
<code>vertical-align</code>	Aligns the text vertically in a container element. Most common values are: <code>top</code> , <code>bottom</code> , and <code>middle</code> .
<code>word-spacing</code>	Adjusts the space between words. Can be the value <code>normal</code> or a length unit.

<http://htmlcheatsheet.com/css/>

F Fonts

FONTS (TYPOGRAPHY)

font-family: font1, font2, font3 ..., genericfont

font-size: size %

REL smaller larger

ABS xx-small x-small small medium large x-large xx-large

font-size-adjust: none number

font-style: normal italic oblique

font-synthesis: none weight style

font-variant: none small-caps

font-weight: normal bold

REL lighter bolder

ABS 100 200 300 400 500 600 700 800 900

font: style variant weight stretch size/ line-height family

caption icon menu message-box small-caption status-bar

GENERIC FONT

serif sans-serif cursive fantasy monospace

EXAMPLE *font-family: Verdana, Arial, sans-serif;*

<https://fonts.googleapis.com/css?family=Open+Sans:300,400|Roboto:400>

TRANSFORMING TEXT

text-transform: **none** **capitalize** **uppercase** **lowercase** **full-width**

WHITE SPACES

white-space: **normal** **pre** **nowrap** **pre-wrap** **pre-line**

tab-size: **8** **number** **size**

LINE BREAKING & WORD BOUNDARIES

line-break: **auto** **loose** **normal** **strict**

word-break: **normal** **keep-all** **break-all**

BREAKING WORDS

hyphens: **none** **manual** **auto**

word-wrap: **normal** **break-word**

overflow-wrap: **normal** **break-word**

ALIGNMENT & JUSTIFICATION

text-align: **left** **right** **center** **justify** **start** **end** **match-parent**

text-align-last: **auto** **start** **end** **left** **right** **center** **justify**

text-justify: **auto** **none** **inter-word** **distribute**

SPACING

word-spacing: **normal** **size**

letter-spacing: **normal** **size**

EDGE EFFECTS

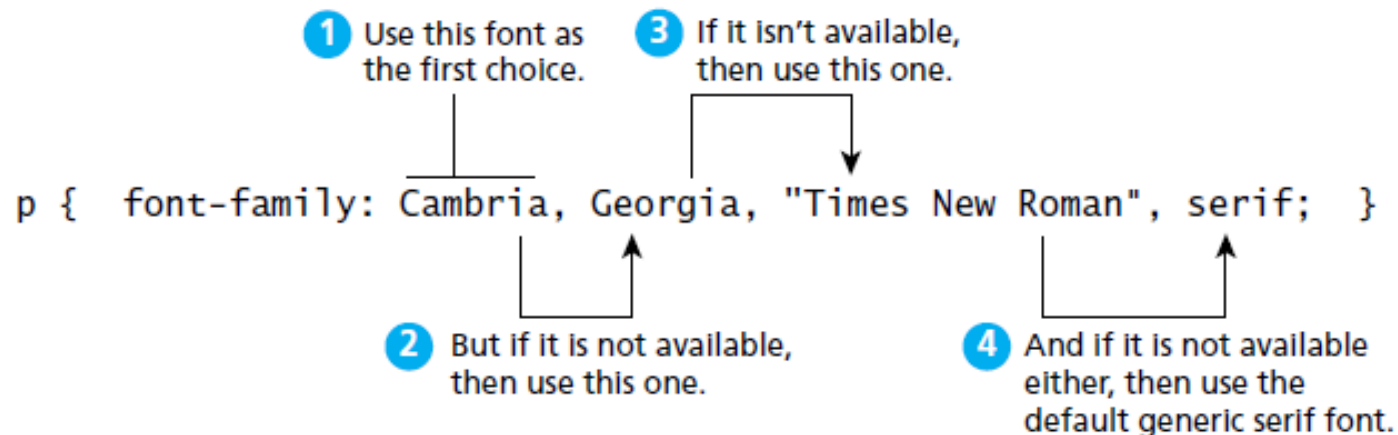
text-indent: **size** **hanging** **each-line**

hanging-punctuation: **none** **first** **last** **force-end** **allow-end**

Specifying font-family

font-family

- You can specify a list of fonts separated by commas so that, if the user does not have your first choice of typeface installed, the browser can try to use an alternative font from the list.
- It is also common to end with a generic font name for that type of font



Specifying font-family

```
<style type="text/css">
  body {
    font-family: Georgia, Times, serif;}
  h1, h2 {
    font-family: Arial, Verdana, sans-serif;}
  .credits {
    font-family: "Courier New", Courier,
      monospace;}
</style>
```

Specifying Typefaces

@font-face: allows you to use a font, even if it is not installed on the computer of the person browsing, by allowing you to specify a path to a copy of the font, which will be downloaded if it is not on the user's machine.

```
@font-face {  
  font-family: 'MyWebFont';  
  src: url('myfont.woff2') format('woff2'),  
       url('myfont.woff') format('woff'),  
       url('myfont.ttf') format('truetype');  
}
```

Custom typefaces

- There are several ways to use fonts other than those listed on the previous slides. (e.g. Google fonts)

<https://fonts.google.com/>

- The open source fonts in the Google Fonts catalog are published under licenses that allow you to use them on any website, whether it's commercial or personal.

Roboto
Christian Robertson (12 styles)



All their equipment
and instruments
are alive.

Mukta Mahee
Ek Type (7 styles)



A red flare
silhouetted the
jagged edge of a
wing.

Open Sans
Steve Matteson (10 styles)



I watched the
storm, so
beautiful yet
terrific.

Font size

font-size :The font-size property enables you to specify a size for the font.

```
body {  
    font-family: Arial, Verdana, sans-serif;  
    font-size: 12px;}  
h1 {  
    font-size: 200%;}  
h2 {  
    font-size: 1.3em;}
```

Font size

PIXELS

TWELVE PIXEL SCALE

h1	24px
h2	18px
h3	14px
body	12px

=

PERCENTAGES

h1	200%
h2	150%
h3	117%
body	75%

=

EMS

h1	1.5em
h2	1.3em
h3	1.17em
body	100%
p	0.75em

SIXTEEN PIXEL SCALE

h1	32px
h2	24px
h3	18px
body	16px

=

h1	200%
h2	150%
h3	133%
body	100%

=

h1	2em
h2	1.5em
h3	1.125em
body	100%
p	1em



CSS selectors

CSS rules

A CSS rule contains two parts: a **selector** and a **declaration**.

SELECTOR



p {

font-family: Arial; }



DECLARATION

Selectors indicate which element the rule applies to.

This rule indicates that all **<p>** elements should be shown in the Arial typeface.

Selectors

When defining CSS rules, you will need to first need to use a selector to tell the browser which elements will be affected.

CSS selectors allow you to select

- individual elements
- multiple HTML elements,
- elements that belong together in some way, or
- elements that are positioned in specific ways in the document hierarchy.

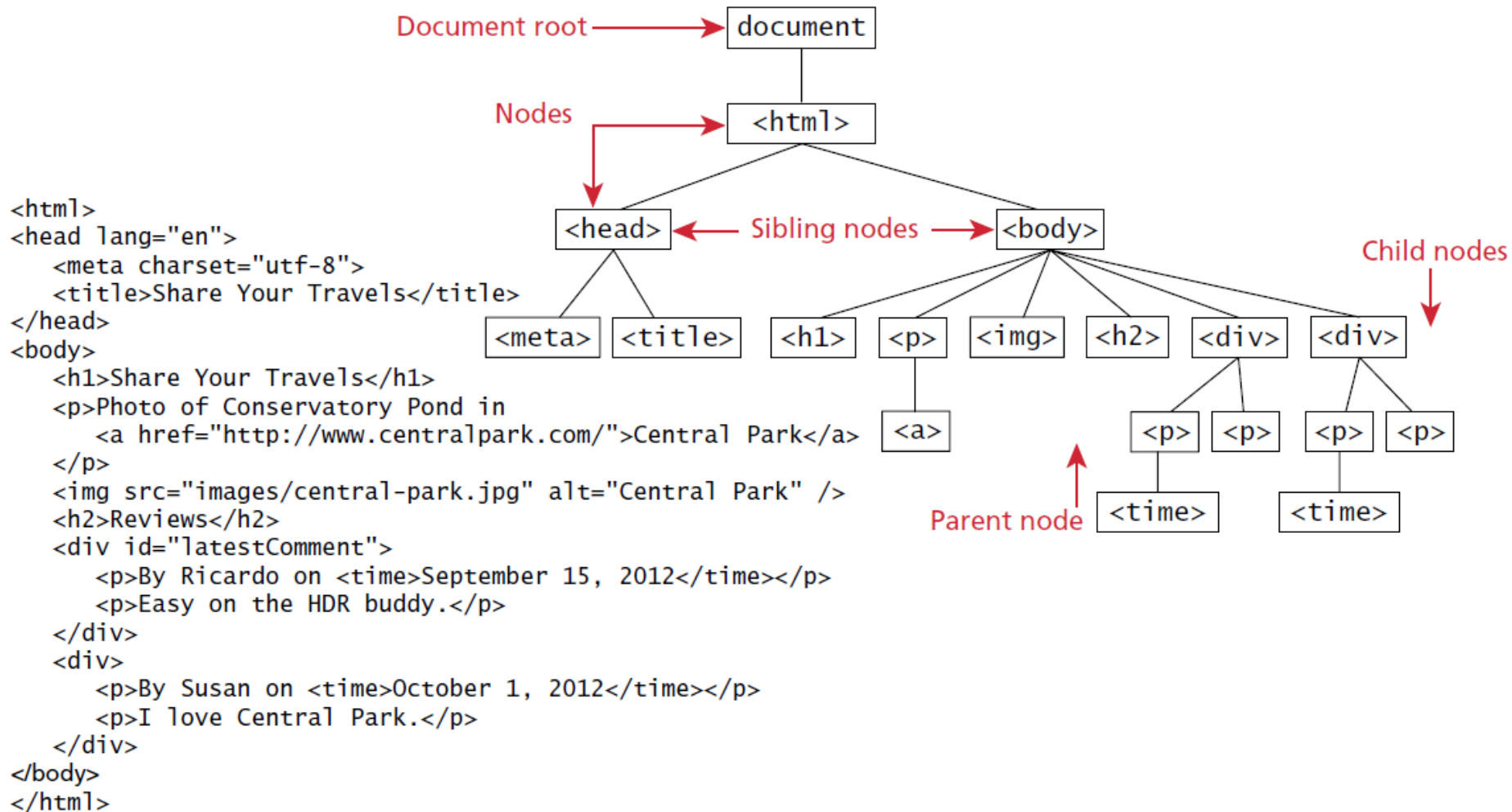
There are a number of different selector types.

DOM

- We need to programmatically access the elements and attributes within the HTML.
- This is accomplished through a programming interface (API) called the **Document Object Model (DOM)**.

DOM: Platform- and language-neutral interface that will allow programs and scripts to dynamically access and update the content, structure and style of documents.

DOM TREE

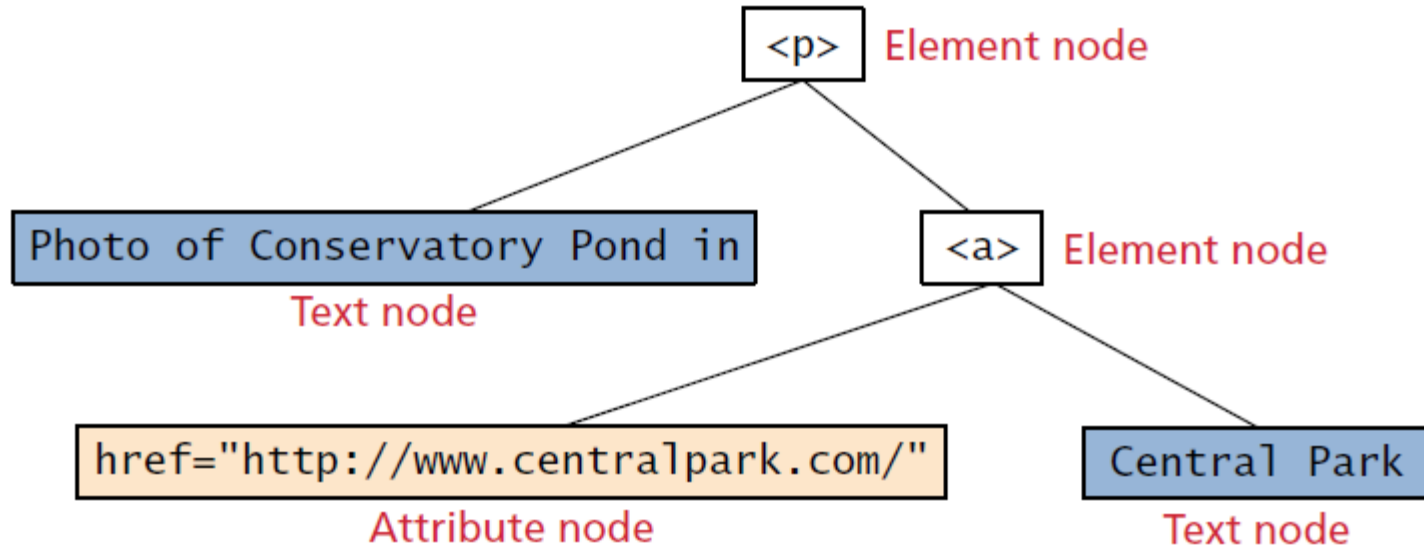


In the DOM, documents have a logical structure which is very much like a tree;

Nodes

- In the DOM, each element within the HTML document is called a **node**. If the DOM is a tree, then each node is an individual branch. There are element nodes, text nodes, and attribute nodes.

```
<p>Photo of Conservatory Pond in  
  <a href="http://www.centralpark.com/">Central Park</a>  
</p>
```



DOM and CSS

- When defining CSS rules, you will need to first use a selector to tell the browser which elements will be affected by the property values.
- CSS selectors allow you to select individual or multiple HTML elements.
- The topic of selectors has become more complicated than it was when we started teaching CSS in the late 1990s.
- There are now a variety of new selectors that are supported by most modern browsers.

Type selectors

- **Type selectors (or element selectors)** select all instances of a given HTML element.

declaration

selector { property: value; property2: value2; } rule

declaration block

selector

em { color: red; }

property value

```
p {
    margin: 5px 0 10px 0;
    font-weight: bold;
    font-family: Arial, Helvetica, sans-serif;
}
```

Type selectors / element selectors

Here is some HTML:

```
1 <p>What color do you like?</p>
2 <div>I like blue.</div>
3 <p>I prefer red!</p>
```

A simple style sheet:

```
1 /* All p elements are red */
2 p {
3     color: red;
4 }
5
6 /* All div elements are blue */
7 div {
8     color: blue;
9 }
```

What color do you like?

I like blue.

I prefer red!

Grouped Selectors

You can select a group of elements by separating the different element names with commas.

This is a sensible way to reduce the size and complexity of your CSS files, by combining multiple identical rules into a single rule.

```
/* commas allow you to group selectors */
p, div, aside {
  margin: 0;
  padding: 0;
}
/* the above single grouped selector is equivalent to the
   following: */
p {
  margin: 0;
  padding: 0;
}
div {
  margin: 0;
  padding: 0;
}
aside {
  margin: 0;
  padding: 0;
}
```

Class Selectors

- A **class selector** allows you to simultaneously target different HTML elements regardless of their position in the document tree.
- If a series of HTML element have been labeled with ***the same class attribute value***, then you can target them for styling by using a class selector, which takes the form: period (.) followed by the class name.

Class Selectors

```

1 <ul>
2   <li class="first done">Create an HTML document</li>
3   <li class="second done">Create a CSS style sheet</li>
4   <li class="third">Link them all together</li>
5 </ul>

```

- ~~Create an HTML document~~
- ~~Create a CSS style sheet~~
- Link them all together

```

1 /* The element with the class "first" is bolded */
2 .first {
3     font-weight: bold;
4 }
5
6 /* All the elements with the class "done" are strike through */
7 .done {
8     text-decoration: line-through;
9 }

```

Id Selectors

- An **id selector** allows you to target a specific element by its id attribute regardless of its type or position.
- If an HTML element has been labeled with an id attribute, then you can target it for styling by using an id selector, which takes the form: pound/hash (#) followed by the id name.
- Note: You should only be using an **id** once per page

Id Selectors

Let's look at a quick example — here is some HTML:

```
1 <p id="polite"> – "Good morning."</p>  
2 <p id="rude"> – "Go away!"</p>
```

A simple style sheet:

```
1 #polite {  
2   font-family: cursive;  
3 }  
4  
5 #rude {  
6   font-family: monospace;  
7   text-transform: uppercase;  
8 }
```

– "Good morning."

– "GO AWAY!"

id versus class Selectors

- id selectors should only be used when referencing a single HTML element since an id attribute can only be assigned to a single HTML element.
- class selectors should be used when (potentially) referencing several related elements.

CSS Diner - Where we feast on CSS Selectors!

CSS Diner

Share

Select the apple directly on a plate



CSS Editor

style.css

HTML Viewer

table.html

```
1 Type in a CSS selector
2 {
3 /* Styles would go here. */
4 }
5
6 /*
7 Type a number to skip to a level.
8 Ex → "5" for level 5
```

```
1 <div class="table">
2   <plate>
3     <bento>
4       <apple />
5     </bento>
6   </plate>
7   <plate>
8     <apple />
```

Level 14 of 32 ✓

< > ≡

Child Selector

Select direct children of an element

A > B

You can select elements that are direct children of other elements. A child element is any element that is nested directly in another element.

Elements that are nested deeper than that are called descendant elements.

Examples

A > B selects all B that are a direct children

A

<https://flukeout.github.io/>

Universal selector (*)

The universal selector (*) is the ultimate joker. It allows selecting all elements in a page. As it is rarely useful to apply a style to every element on a page, it is often used in combination with other selectors

```
*  
{ margin: 0;  
  padding: 0; }
```

```
*[lang=fr]{color:blue;}  
/* sets color blue for texts written in  
french language */
```

Because each browser uses a different stylesheet, certain elements (submit buttons, links, etc) don't always have consistent styling across all browsers. By resetting the margin and the padding of every HTML element to 0px, the Universal Reset helps to normalize the format of a page without being too heavy.

Combinators and groups of selectors

Select elements based on their ancestors, descendants, or siblings

- **Combinators** allows you to select elements based on their ancestors, descendants, or siblings.
- That is, it selects elements based on their context or their relation to other elements in the document tree.

Combinators and groups of selectors

Name	Syntax	Selects
Group of selectors	A, B	Any element matching A and/or B (see Group of selectors on one rule , below - Group of Selectors is not considered to be a combinator).
Descendant selector	A B	Any element matching B that is a <i>descendant</i> of an element matching A (that is: a child, or a child of a child, <i>etc.</i>)
Child selector	A > B	Any element matching B that is a <i>direct child</i> of an element matching A.
Adjacent sibling selector	A + B	Any element matching B that is the next <i>sibling</i> of an element matching A (that is: the next child of the same parent.)
General sibling selector	A ~ B	Any element matching B that is one of the next <i>siblings</i> of an element matching A (that is: one of the next children of the same parent.)

Descendant Selector

Selects all elements that are contained within another element

- A **descendant selector** matches all elements that are contained within another element. The character used to indicate descendant selection is the space character.

context selected element

`div p { ... }`



Selects a `<p>` element
somewhere
within a `<div>` element

`#main div p:first-child { ... }`



Selects the first `<p>` element
somewhere within a `<div>` element
that is somewhere within an element
with an `id="main"`

Combinators in Action

```

<body>
  <nav>
    <ul>
      <li><a href="#">Canada</a></li>
      <li><a href="#">Germany</a></li>
      <li><a href="#">United States</a></li>
    </ul>
  </nav>
  <div id="main">
    Comments as of <time>November 15, 2015</time>
    <div>
      <p>By Ricardo on <time>September 15, 2015</time></p>
      <p>Easy on the HDR buddy.</p>
    </div>
    <hr/>
    <div>
      <p>By Susan on <time>October 1, 2015</time></p>
      <p>I love Central Park.</p>
    </div>
    <hr/>
  </div>
  <footer>
    <ul>
      <li><a href="#">Home</a> | </li>
      <li><a href="#">Browse</a> | </li>
    </ul>
  </footer>
</body>

```

ul a:link { color: blue; }

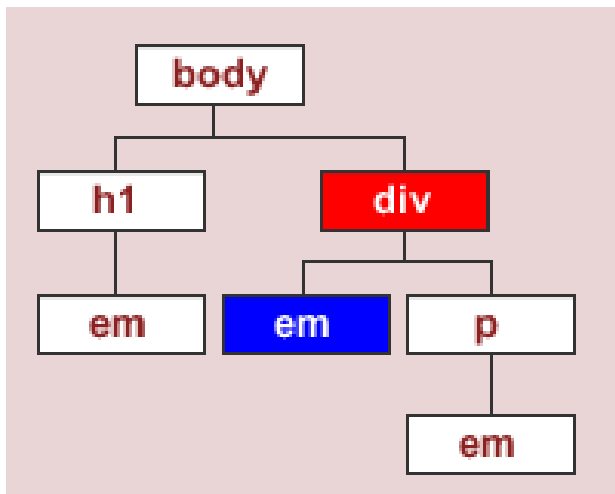
#main>time { color: purple; }

#main div p:first-child {
color: green;
}

#main time { color: red; }

Child selectors

```
<body>
<h1>Heading <em>text</em></h1>
<div>
This is some <em>text</em>
<p>This is a paragraph of <em>text</em></p>
</div>
</body>
```



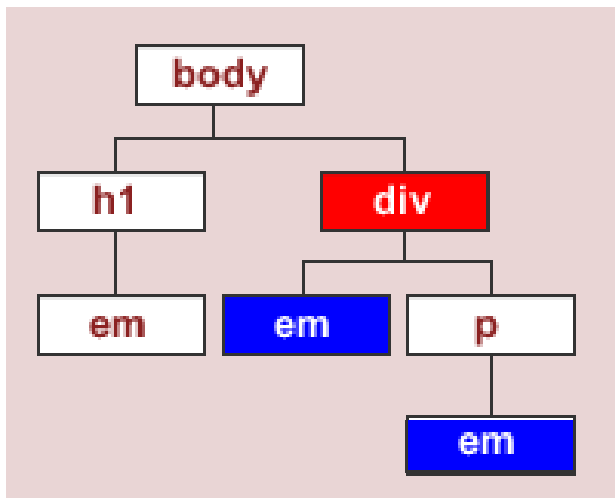
```
div > em { color: blue; }
```

OR

```
div>em { color: blue; }
```


Descendant selectors

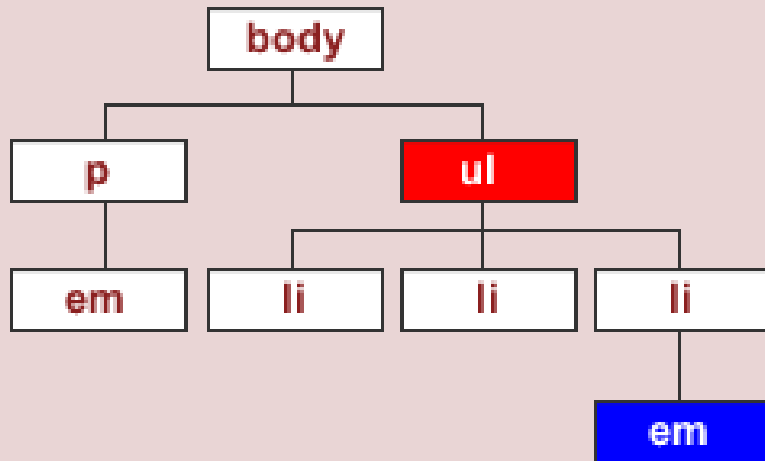
```
<body>
<h1>Heading <em>text</em></h1>
<div>
This is some <em>text</em>
<p>This is a paragraph of <em>text</em></p>
</div>
</body>
```



```
div em { color: blue; }
```

Descendant selectors

```
<body>
<p>Lorem ipsum dolor <em>sit</em> amet.</p>
<ul>
<li>item 1</li>
<li>item 2</li>
<li><em>item 3</em></li>
</ul>
</body>
```



```
ul em {color: blue; }
```

Pseudo Selectors

Select something that does not exist explicitly as an element

- A **pseudo-element selector** is a way to select something that does not exist explicitly as an element in the HTML document tree but which is still a recognizable selectable object.
- A **pseudo-class selector** does apply to an HTML element, but targets either a particular state or, in CSS3, a variety of family relationships.

Pseudo elements

Pseudo-elements are keywords preceded by two colons (::), that can be added to the end of selectors to select a certain part of an element.

Value	Description
::first-line	Use this element to add special styles to the first line of the text in a selector.
::first-letter	Use this element to add special style to the first letter of the text in a selector.
::before	Use this element to insert some content before an element.
::after	Use this element to insert some content after an element.

https://www.tutorialspoint.com/css/css_pseudo_elements.htm

Pseudo elements

```
<html>
  <head>
    <style type="text/css">
      p::first-letter { font-size: 5em; }

    </style>
  </head>
  <body>

    <p>The first character of this paragraph will be 5em big
      paragraph will remain normal. This example shows how
      characters of any HTML element.</p>

  </body>
</html>
```



The first character of this paragraph will be 5em big as defined in the CSS rule above. Rest of the characters in this paragraph will remain normal. This example shows how to use :first-letter pseudo element to give effect to the first characters of any HTML element.

Pseudo elements

```
<html>
  <head>
    <style type="text/css">
      p::before
      {
        content: url(/images/bullet.gif)
      }
    </style>
  </head>
  <body>
    <p> This line will be preceded by a bullet.</p>
    <p> This line will be preceded by a bullet.</p>
    <p> This line will be preceded by a bullet.</p>
  </body>
</html>
```

- This line will be preceded by a bullet.
- This line will be preceded by a bullet.
- This line will be preceded by a bullet.

Pseudo classes

A CSS pseudo-class is a keyword, preceded by a colon (:), added to the end of selectors to specify you want to style the selected elements, and only when they are in certain state.

Pseudo classes

Selector	Example	Example description
<u>:active</u>	a:active	Selects the active link
<u>:checked</u>	input:checked	Selects every checked <input> element
<u>:disabled</u>	input:disabled	Selects every disabled <input> element
<u>:empty</u>	p:empty	Selects every <p> element that has no children
<u>:enabled</u>	input:enabled	Selects every enabled <input> element
<u>:first-child</u>	p:first-child	Selects every <p> elements that is the first child of its parent
<u>:first-of-type</u>	p:first-of-type	Selects every <p> element that is the first <p> element of its parent
<u>:focus</u>	input:focus	Selects the <input> element that has focus
<u>:hover</u>	a:hover	Selects links on mouse over
<u>:in-range</u>	input:in-range	Selects <input> elements with a value within a specified range
<u>:invalid</u>	input:invalid	Selects all <input> elements with an invalid value
<u>:lang(<i>language</i>)</u>	p:lang(it)	Selects every <p> element with a lang attribute value starting with "it"
<u>:last-child</u>	p:last-child	Selects every <p> elements that is the last child of its parent
<u>:last-of-type</u>	p:last-of-type	Selects every <p> element that is the last <p> element of its parent

https://www.w3schools.com/css/css_pseudo_classes.asp

<u>:link</u>	a:link	Selects all unvisited links
<u>:not(selector)</u>	:not(p)	Selects every element that is not a <p> element
<u>:nth-child(n)</u>	p:nth-child(2)	Selects every <p> element that is the second child of its parent
<u>:nth-last-child(n)</u>	p:nth-last-child(2)	Selects every <p> element that is the second child of its parent, counting from the last child
<u>:nth-last-of-type(n)</u>	p:nth-last-of-type(2)	Selects every <p> element that is the second <p> element of its parent, counting from the last child
<u>:nth-of-type(n)</u>	p:nth-of-type(2)	Selects every <p> element that is the second <p> element of its parent
<u>:only-of-type</u>	p:only-of-type	Selects every <p> element that is the only <p> element of its parent
<u>:only-child</u>	p:only-child	Selects every <p> element that is the only child of its parent
<u>:optional</u>	input:optional	Selects <input> elements with no "required" attribute

<u>:out-of-range</u>	input:out-of-range	Selects <input> elements with a value outside a specified range
<u>:read-only</u>	input:read-only	Selects <input> elements with a "readonly" attribute specified
<u>:read-write</u>	input:read-write	Selects <input> elements with no "readonly" attribute
<u>:required</u>	input:required	Selects <input> elements with a "required" attribute specified
<u>:root</u>	root	Selects the document's root element
<u>:target</u>	#news:target	Selects the current active #news element (clicked on a URL containing that anchor name)
<u>:valid</u>	input:valid	Selects all <input> elements with a valid value
<u>:visited</u>	a:visited	Selects all visited links

Pseudo classes

It can be used to:

- Style an element when a user mouses over it
- Style visited and unvisited links differently
- Style an element when it gets focus

```
body {background-color:black; color:white}
```

```
a:link      {text-decoration: underline; color: yellow;}
a:visited   {text-decoration: underline; color: orange;}
a:hover     {text-decoration: none; font-weight: bold;}
a:focus     {background-color: lightgray; }
```

Links are an important part of any web page. To learn more about links visit the [W3C](#) website.

- [Canada](#)
- [Germany](#)
- [United States](#)

:last-child()

```

1 <div>
2   <p>This text isn't selected.</p>
3   <p>This text is selected!</p>
4 </div>
5
6 <div>
7   <p>This text isn't selected.</p>
8   <h2>This text isn't selected: it's not a `p`.</h2>
9 </div>

```

This text isn't selected.

This text is selected!

This text isn't selected.

This text isn't selected: it's not a `p`.

```

1 p:last-child {
2   color: lime;
3   background-color: black;
4   padding: 5px;
5 }

```

:nth-child()

The `:nth-child(n)` selector matches every element that is the *n*th child, regardless of type, of its parent.

HTML

```

1
2 <p>Children 1, 3, 5, and 7 are selected.</p>
3 <div>
4   <span>Span 1!</span>
5   <span>Span 2</span>
6   <span>Span 3!</span>
7   <span>Span 4</span>
8   <span>Span 5!</span>
9   <span>Span 6</span>
10  <span>Span 7!</span>
11 </div>

```

CSS

```

1
2 div span:nth-child(2n+1)
3 {
4   background-color: lime;
5 }

```

Children 1, 3, 5, and 7 are selected.

Span 1! Span 2 Span 3! Span 4 Span 5! Span 6 Span 7!

Attribute Selectors

- An **attribute selector** provides a way to select HTML elements by either the presence of an element attribute or by the value of an attribute.
- This can be a very powerful technique, but because of uneven support by some of the browsers, not all web authors have used them.
- Attribute selectors can be a very helpful technique in the styling of hyperlinks and images.

Attribute Selectors (CSS3)

Selector	Matches	Example
[]	A specific attribute.	[title] Matches any element with a title attribute
[=]	A specific attribute with a specific value.	a[title="posts from this country"] Matches any <a> element whose title attribute is exactly "posts from this country"
[~=]	A specific attribute whose value matches at least one of the words in a space-delimited list of words.	[title~="Countries"] Matches any title attribute that contains the word "Countries"
[^=]	A specific attribute whose value begins with a specified value.	a[href^="mailto"] Matches any <a> element whose href attribute begins with "mailto"
[*=]	A specific attribute whose value contains a substring.	img[src*="flag"] Matches any element whose src attribute contains somewhere within it the text "flag"
[\$=]	A specific attribute whose value ends with a specified value.	a[href\$=".pdf"] Matches any <a> element whose href attribute ends with the text ".pdf"

```

1 Ingredients for my recipe: <i lang="fr-FR">Poulet basquaise</i>
2 <ul>
3   <li data-quantity="1kg" data-vegetable>Tomatoes</li>
4   <li data-quantity="3" data-vegetable>Onions</li>
5   <li data-quantity="3" data-vegetable>Garlic</li>
6   <li data-quantity="700g" data-vegetable="not spicy like chili">Red pepper</li>
7   <li data-quantity="2kg" data-meat>Chicken</li>
8   <li data-quantity="optional 150g" data-meat>Bacon bits</li>
9   <li data-quantity="optional 10ml" data-vegetable="liquid">Olive oil</li>
10  <li data-quantity="25cl" data-vegetable="liquid">White wine</li>
11 </ul>

```

```

/* All elements with the attribute "data-vegetable"
are given green text */

```

```
[data-vegetable] { color: green; }
```

```

/* All elements with the attribute "data-vegetable"
with the exact value "liquid" are given a golden
background color */

```

```
[data-vegetable="liquid"] { background-color:
goldenrod; }
```

```

/* All elements with the attribute "data-vegetable",
containing the value "spicy", even among others, are
given a red text color */

```

```
[data-vegetable~="spicy"] { color: red; }
```

Ingredients for my recipe: *Poulet basquaise*

- Tomatoes
- Onions
- Garlic
- Red pepper
- Chicken
- Bacon bits
- Olive oil
- White wine


```

1 Ingredients for my recipe: <i lang="fr-FR">Poulet basquaise</i>
2 <ul>
3   <li data-quantity="1kg" data-vegetable>Tomatoes</li>
4   <li data-quantity="3" data-vegetable>Onions</li>
5   <li data-quantity="3" data-vegetable>Garlic</li>
6   <li data-quantity="700g" data-vegetable="not spicy like chili">Red pepper</li>
7   <li data-quantity="2kg" data-meat>Chicken</li>
8   <li data-quantity="optional 150g" data-meat>Bacon bits</li>
9   <li data-quantity="optional 10ml" data-vegetable="liquid">Olive oil</li>
10  <li data-quantity="25cl" data-vegetable="liquid">White wine</li>
11 </ul>

```

```

/* Classic usage for language selection */
[lang|=fr] { font-weight: bold; }

/* All elements with the attribute "data-quantity", for
which the value starts with "optional" */
[data-quantity^="optional"] { opacity: 0.5; }

/* All elements with the attribute "data-quantity", for
which the value ends with "kg" */
[data-quantity$="kg"] { font-weight: bold; }

/* All elements with the attribute "data-vegetable"
containing the value "not spicy" are turned back to green
*/
[data-vegetable*="not spicy"] { color: green; }

```

Ingredients for my recipe: *Poulet basquaise*

- **Tomatoes**
- Onions
- Garlic
- **Red pepper**
- **Chicken**
- Bacon bits
- Olive oil
- White wine

<https://www.w3.org/TR/2018/CR-selectors-3-20180130/>

Pattern	Represents	Description	Level
*	any element	Universal selector	2
E	an element of type E	Type selector	1
E[foo]	an E element with a "foo" attribute	Attribute selectors	2
E[foo="bar"]	an E element whose "foo" attribute value is exactly equal to "bar"	Attribute selectors	2
E[foo~="bar"]	an E element whose "foo" attribute value is a list of whitespace-separated values, one of which is exactly equal to "bar"	Attribute selectors	2
E[foo^="bar"]	an E element whose "foo" attribute value begins exactly with the string "bar"	Attribute selectors	3
E[foo\$="bar"]	an E element whose "foo" attribute value ends exactly with the string "bar"	Attribute selectors	3
E[foo*="bar"]	an E element whose "foo" attribute value contains the substring "bar"	Attribute selectors	3
E[foo ="en"]	an E element whose "foo" attribute has a hyphen-separated list of values beginning (from the left) with "en"	Attribute selectors	2
E:root	an E element, root of the document	Structural pseudo-classes	3

<https://www.w3.org/TR/2018/CR-selectors-3-20180130/>

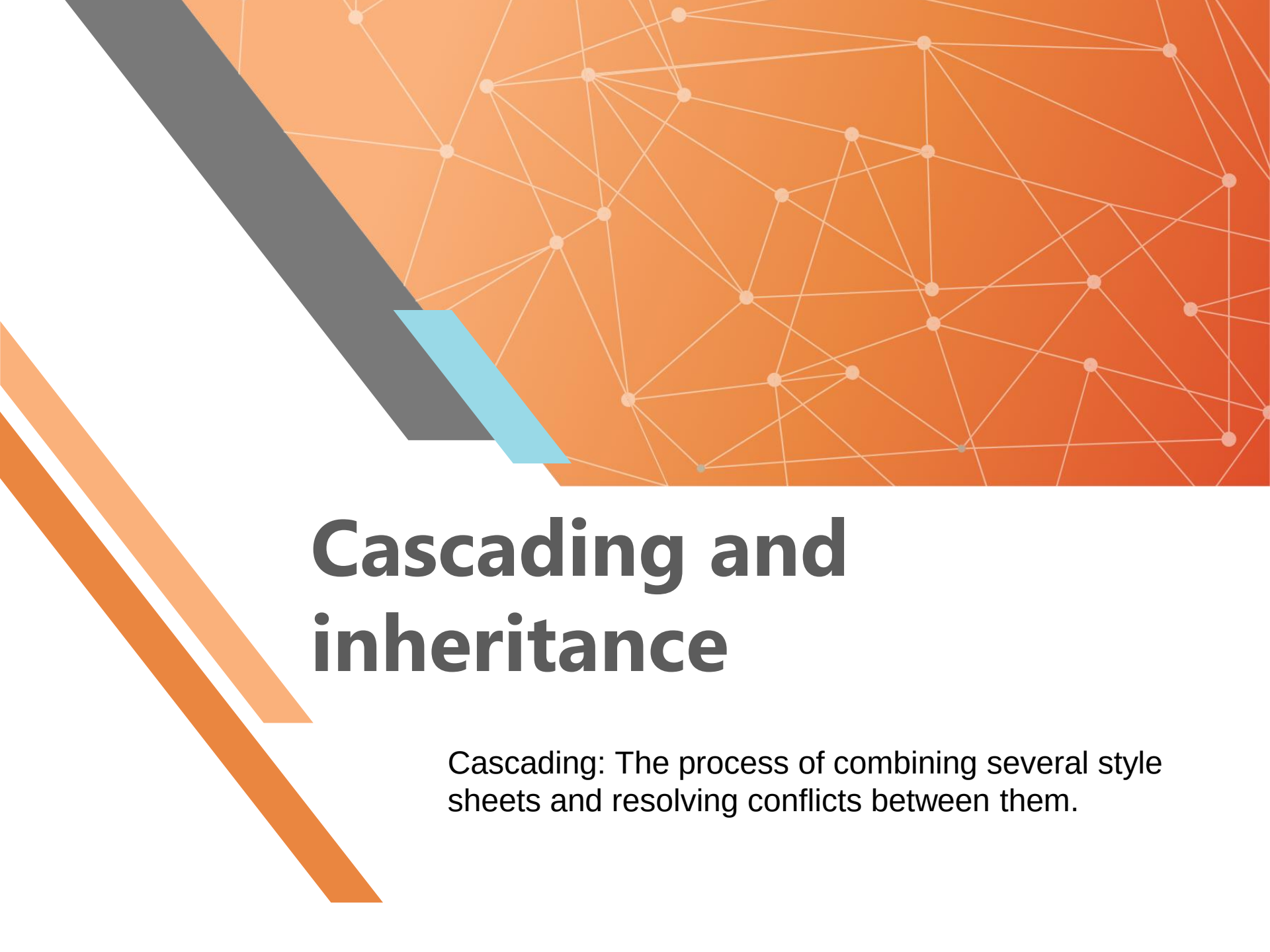
E:nth-child(n)	an E element, the n-th child of its parent	Structural pseudo-classes	3
E:nth-last-child(n)	an E element, the n-th child of its parent, counting from the last one	Structural pseudo-classes	3
E:nth-of-type(n)	an E element, the n-th sibling of its type	Structural pseudo-classes	3
E:nth-last-of-type(n)	an E element, the n-th sibling of its type, counting from the last one	Structural pseudo-classes	3
E:first-child	an E element, first child of its parent	Structural pseudo-classes	2
E:last-child	an E element, last child of its parent	Structural pseudo-classes	3
E:first-of-type	an E element, first sibling of its type	Structural pseudo-classes	3
E:last-of-type	an E element, last sibling of its type	Structural pseudo-classes	3
E:only-child	an E element, only child of its parent	Structural pseudo-classes	3

<https://www.w3.org/TR/2018/CR-selectors-3-20180130/>

E:only-of-type	an E element, only sibling of its type	Structural pseudo-classes	3
E:empty	an E element that has no children (including text nodes)	Structural pseudo-classes	3
E:link	an E element being the source anchor of a hyperlink of which the target is not yet visited (:link) or already visited (:visited)	The link pseudo-classes	1
E:visited			
E:active	an E element during certain user actions	The user action pseudo-classes	1 and 2
E:hover			
E:focus			
E:target	an E element being the target of the referring URI	The target pseudo-class	3
E:lang(fr)	an element of type E in language "fr" (the document language specifies how language is determined)	The :lang() pseudo-class	2
E:enabled	a user interface element E which is enabled or disabled	The UI element states pseudo-classes	3
E:disabled			
E:checked	a user interface element E which is checked (for instance a radio-button or checkbox)	The UI element states pseudo-classes	3

<https://www.w3.org/TR/2018/CR-selectors-3-20180130/>

E::first-line	the first formatted line of an E element	The ::first-line pseudo-element	1
E::first-letter	the first formatted letter of an E element	The ::first-letter pseudo-element	1
E::before	generated content before an E element	The ::before pseudo-element	2
E::after	generated content after an E element	The ::after pseudo-element	2
E.warning	an E element whose class is "warning" (the document language specifies how class is determined).	Class selectors	1
E#myid	an E element with ID equal to "myid".	ID selectors	1
E:not(s)	an E element that does not match simple selector s	Negation pseudo-class	3
E F	an F element descendant of an E element	Descendant combinator	1
E > F	an F element child of an E element	Child combinator	2
E + F	an F element immediately preceded by an E element	Next-sibling combinator	2
E ~ F	an F element preceded by an E element	Subsequent-sibling combinator	3

The background features a complex network of white lines connecting small dots on an orange gradient. On the left, there are several diagonal stripes in shades of orange and grey, with a small light blue parallelogram overlapping the grey one.

Cascading and inheritance

Cascading: The process of combining several style sheets and resolving conflicts between them.

Why Conflict Happens

Because

- there are three different types of style sheets (author-created, user-defined, and the default browser style sheet),
- author-created stylesheets can define multiple rules for the same HTML element,

CSS has a system to help the browser determine how to display elements when different style rules conflict.

Cascade Principles

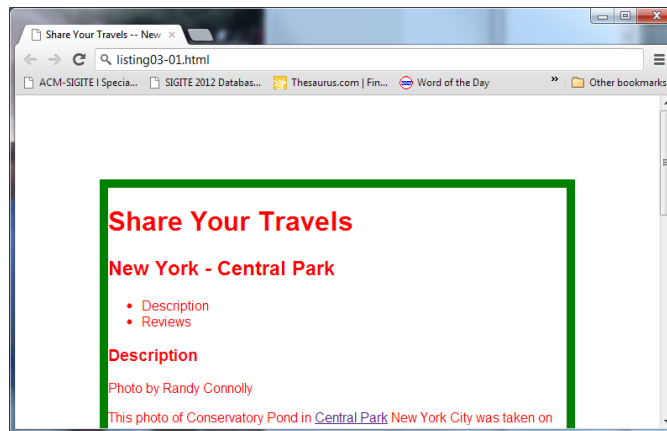
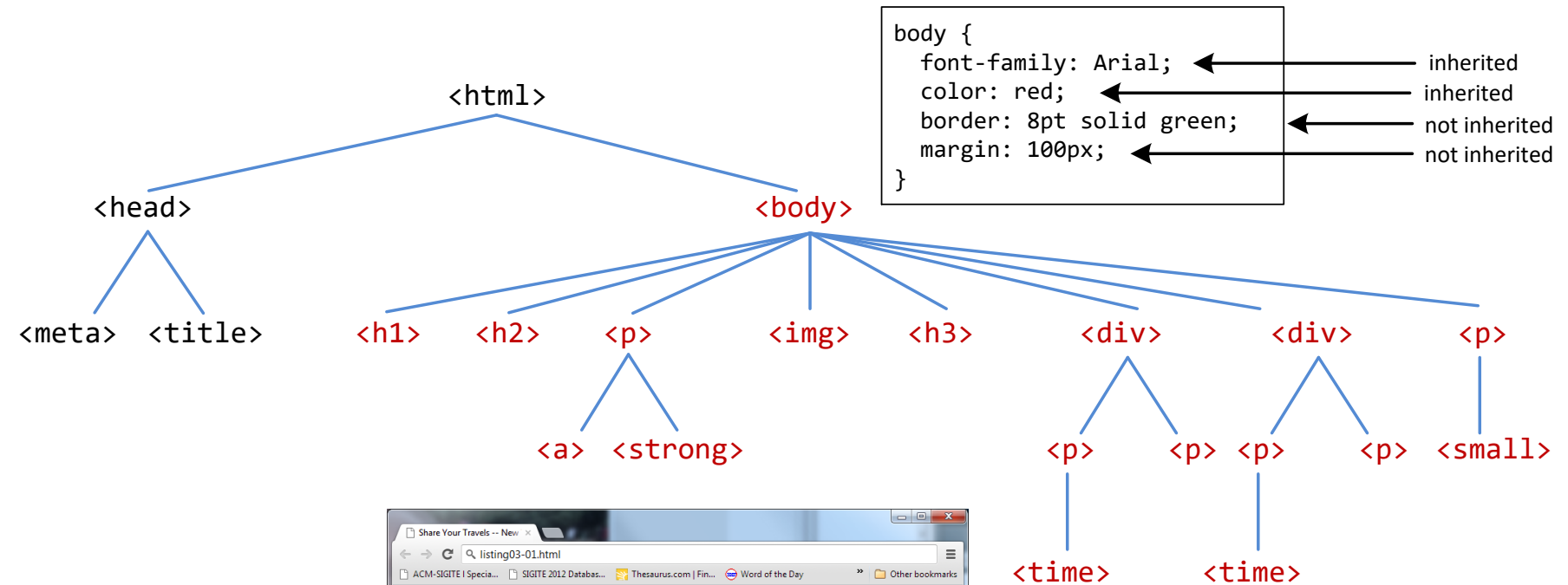
CSS uses the following cascade principles to help it deal with conflicts:

- **inheritance,**
- **specificity,**
- **location**

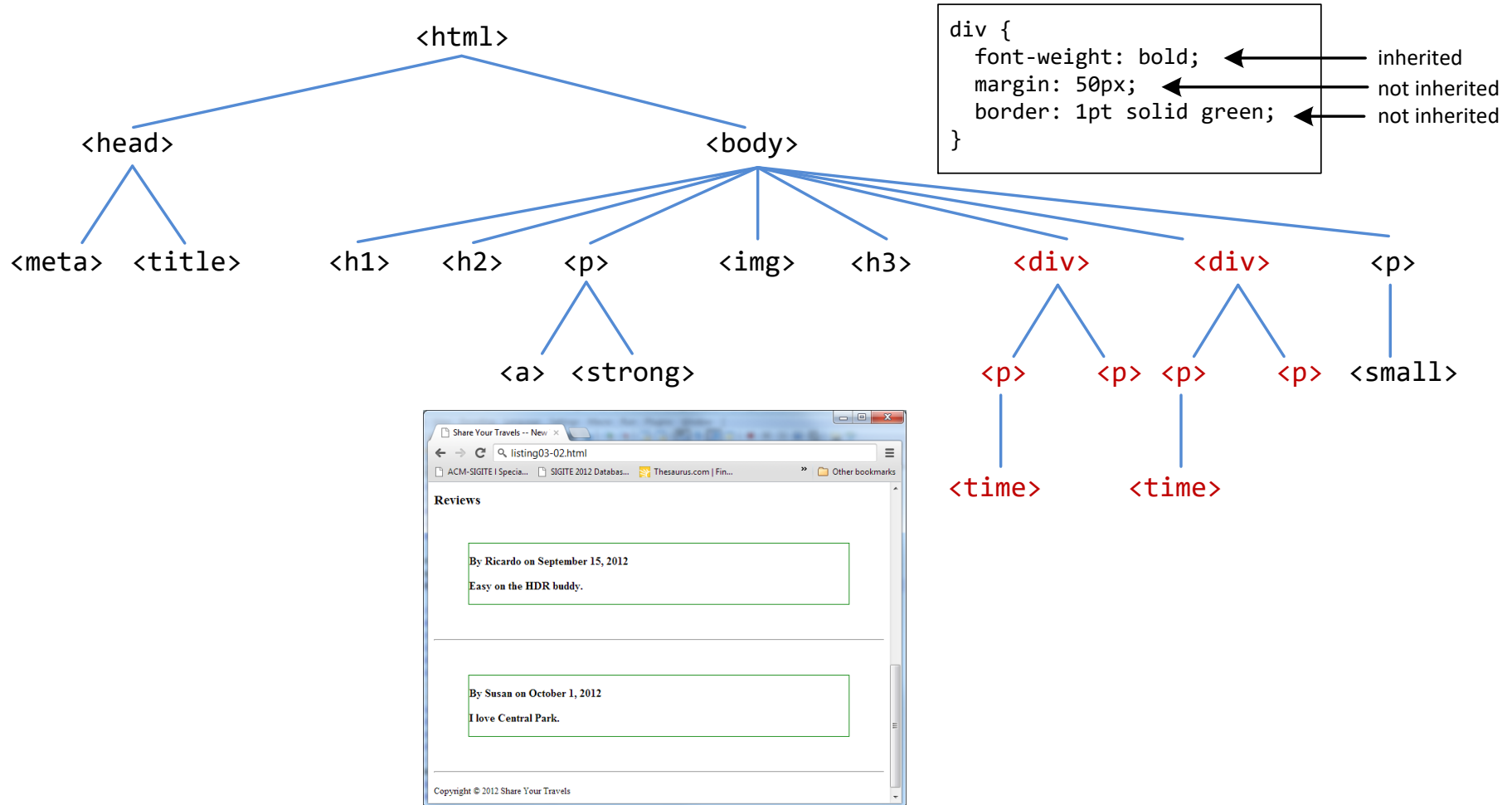
Inheritance

- Many (but not all) CSS properties affect not only themselves but their descendants as well.
- Font, color, list, and text properties **are inheritable**.
- Layout, sizing, border, background and spacing properties **are not**.

Inheritance

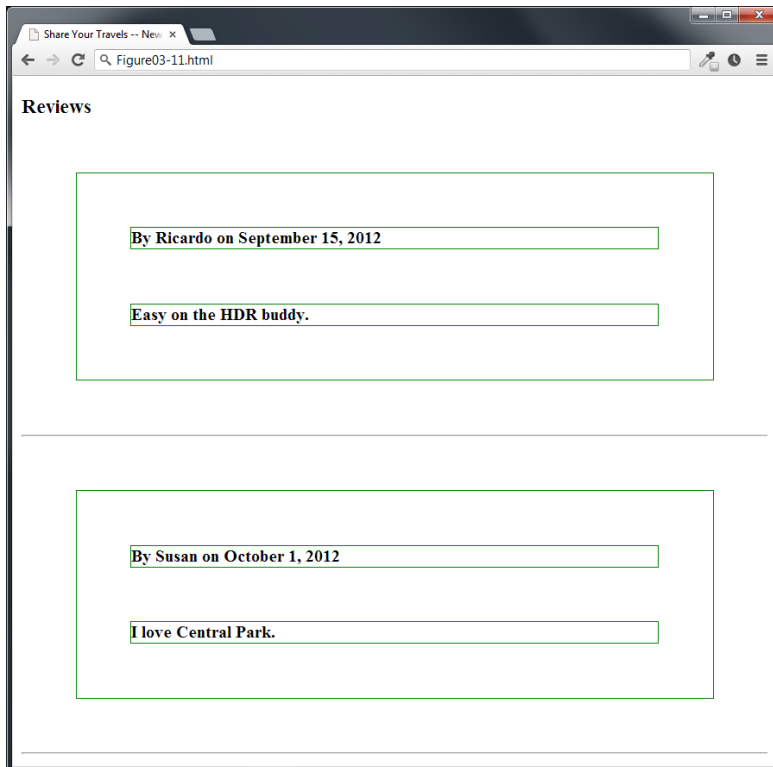


Inheritance



How to force inheritance

It is possible to tell elements to inherit properties that are normally not inheritable, using *inherit* value.



```
div {
  font-weight: bold;
  margin: 50px;
  border: 1pt solid green;
}
p {
  border: inherit;
  margin: inherit;
}

<h3>Reviews</h3>
<div>
  <p>By Ricardo on <time>September 15, 2012</time></p>
  <p>Easy on the HDR buddy.</p>
</div>
<hr/>

<div>
  <p>By Susan on <time>October 1, 2012</time></p>
  <p>I love Central Park.</p>
</div>
<hr/>
```

Specificity

- **Specificity** is how the browser determines which style rule takes precedence when more than one style rule could be applied to the same element.
- The more *specific* the selector, the more it takes precedence (i.e., overrides the previous definition).

Specificity

- The way that specificity works in the browser is that the browser assigns a weight to each style rule.
- When several rules apply, the one with the greatest weight takes precedence.

Recommended video

CSS Basics (Part6) — Specificity

<https://www.youtube.com/watch?v=fy07HYm-geM>



Specificity Algorithm

The algorithm that is used to determine specificity is :

- First count 1 if the declaration is from a 'style' attribute in the HTML, 0 otherwise (let that value = a).
- Count the number of ID attributes in the selector (let that value = b).
- Count the number of other attributes and pseudo-classes in the selector (let that value = c).
- Count the number of element names and pseudo-elements in the selector (let that value = d).
- Finally, concatenate the four numbers $a+b+c+d$ together to calculate the selector's specificity.

Specificity Algorithm

The algorithm that is used to determine specificity is :

<code><tag style="color: red"></code>	1000
<code>body .example</code>	0011
<code>body .example strong</code>	0012
<code>div#first</code>	0101
<code>div#first .error</code>	0111
<code>#footer .twitter a</code>	0111
<code>#footer .twitter a:hover</code>	0121
<code>body aside#left div#cart strong.price</code>	0214

Specificity Algorithm

element selector	<pre>div { color: green; }</pre>	Specificity Value 0001
1 overrides		
descendant selector (elements only)	<pre>div form { color: orange; }</pre>	0002
2 overrides		
class and attribute selectors	<pre>.example { color: blue; } a[href\$=".pdf"] { color: blue; }</pre>	0010
3 overrides		
id selector	<pre>#firstExample { color: magenta; }</pre>	0100
4 overrides		
id + additional selectors	<pre>div #firstExample { color: grey; }</pre>	0101
5 overrides		
inline style attribute	<pre><div style="color: red;"></pre>	1000

A higher specificity value overrides lower specificity values.

Specificity calculator

- <https://specificity.keegan.st/>

```
li:first-child h2 .title
```

0

Inline styles

0

IDs

2

Classes, attributes
and pseudo-classes

2

Elements and
pseudo-elements

+ Duplicate

```
#nav .selected > a:hover
```

0

Inline styles

1

IDs

2

Classes, attributes
and pseudo-classes

1

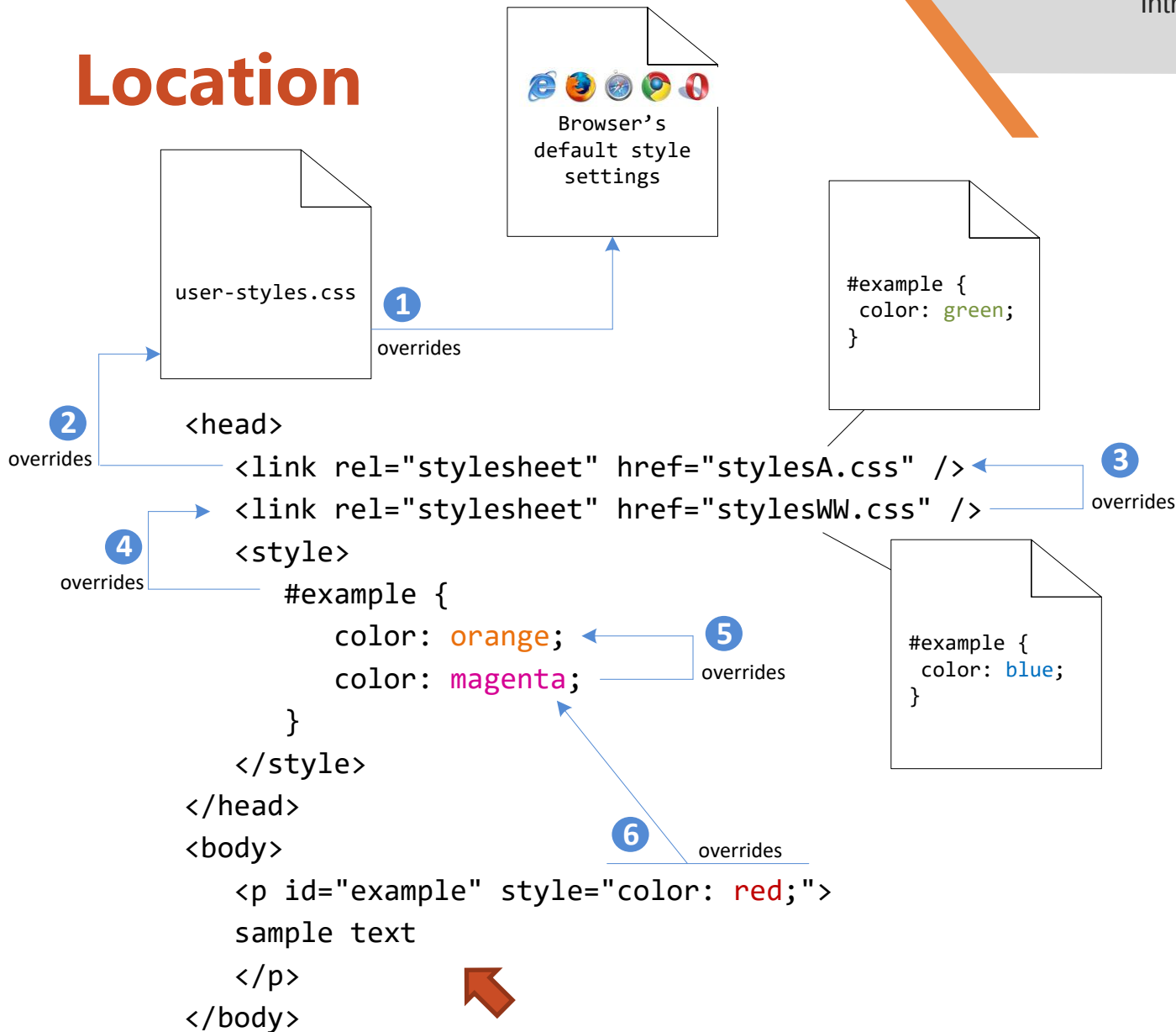
Elements and
pseudo-elements

+ Duplicate

Location

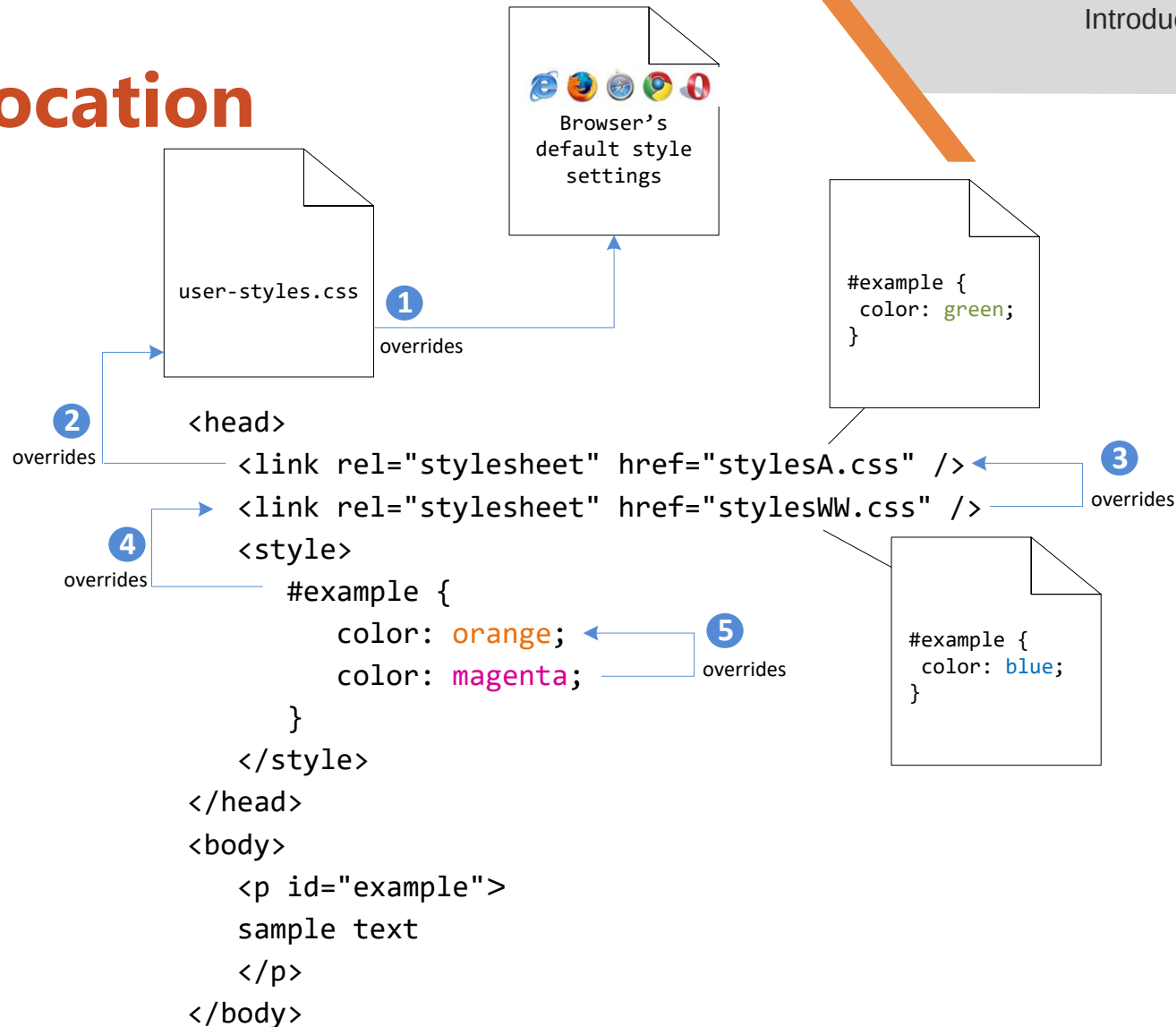
- When inheritance and specificity cannot determine style precedence, the principle of **location** will be used.
- The principle of location is that when rules have the same specificity, then the latest are given more weight.

Location



Can you guess what will be the color of the sample text ?

Location



What color would the sample text be if there wasn't an inline style definition?

Location

There's always an exception

- There is one exception to the principle of location.
- If a property is marked with **!important** in an author-created style rule, then it will override any other author-created style regardless of its location.

```
p.warning {color:red ! important}
```



Layout

Beginning Graphic Design: Layout & Composition

Layout & Composition





The BOX model

The Box Model

- In CSS, all HTML elements exist within an **element box**.
- It is absolutely essential that you familiarize yourself with the terminology and relationship of the CSS properties within the element box.

Block-level and Inline Elements

- Within the CSS Box model, there are two categories of elements
 - block-level
 - inline
- Block-level elements create boxes that are a part of a pages layout
 - this category includes articles, sections, paragraphs, headers, footers and more
- Inline elements are used to format content
 - this category includes emphasis and boldface

BLOCK LEVEL ELEMENT EXAMPLES

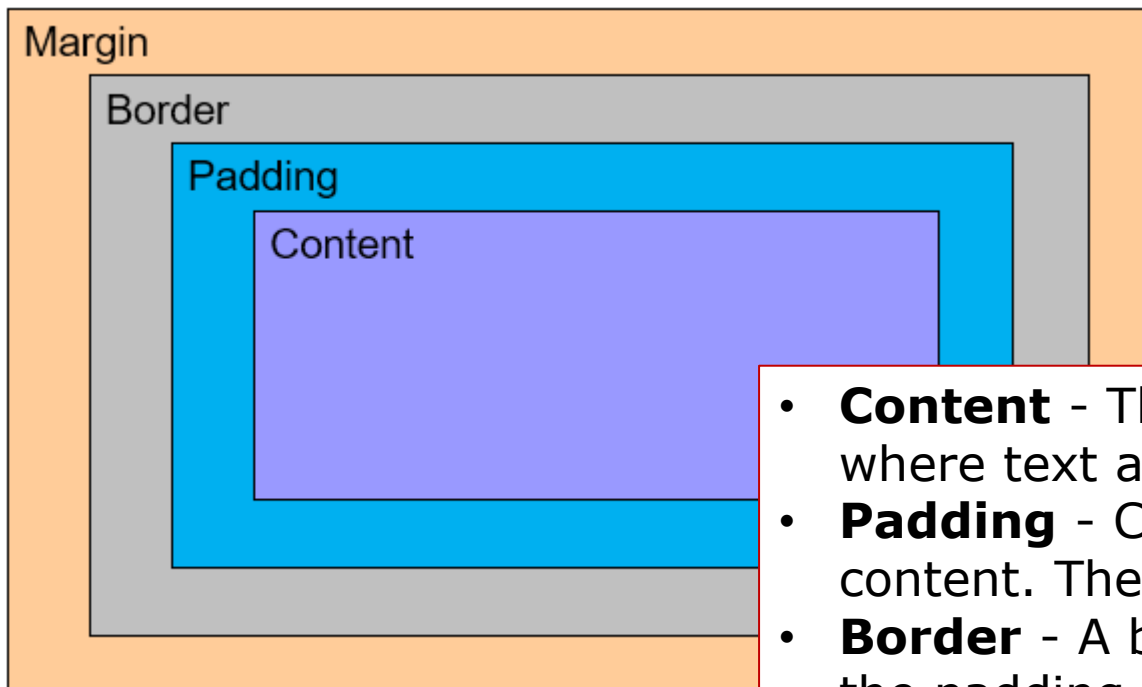
- `<div>`
- `<p>`
- `<ul>`, `<ol>`, `<dl>`
- `<h1>` - `<h6>`
- `<form>`

INLINE ELEMENT EXAMPLES

- `<img>`
- `<a>`
- `<strong>`
- `<em>`
- `<input>`

Box Components

The CSS box model is essentially a box that wraps around every HTML element. It consists of: margins, borders, padding, and the actual content.



- **Content** - The content of the box, where text and images appear
- **Padding** - Clears an area around the content. The padding is transparent
- **Border** - A border that goes around the padding and content
- **Margin** - Clears an area outside the border. The margin is transparent

Introducing the `<div>` Element

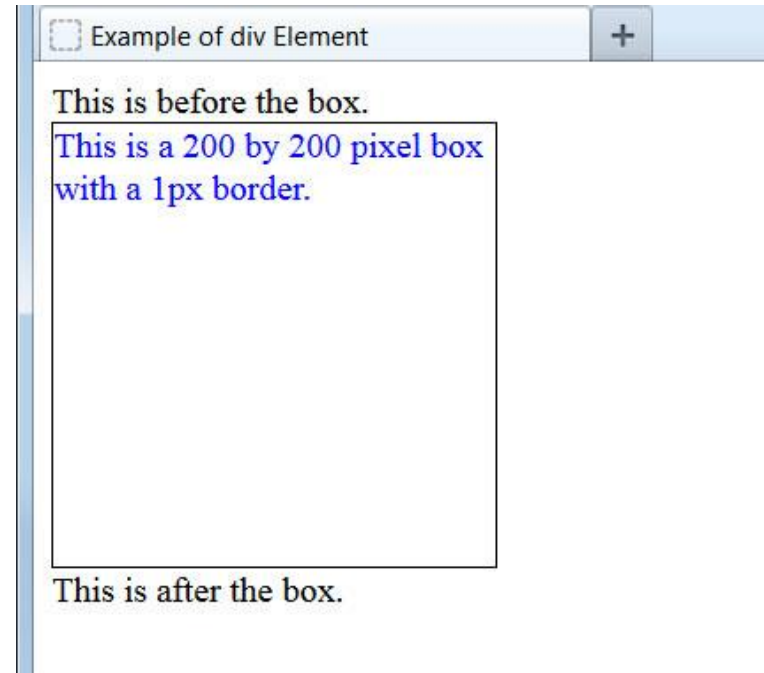
The `<div>` ("division") element groups other elements on the screen.

- By setting the **width** and **height** attributes via CSS, we can reserve a precise amount of space on our page for specific content.
- The actual content is nested and contained within the opening `<div>` and closing `</div>` tags.
- When we apply CSS styling directly to the `<div>` element, all the elements contained within that `<div>` will inherit that style.
- By using multiple `<div>` elements as building blocks, we can design an entire web page layout.

Example <div> Element

Let's create a box on the screen and add a border around it:

```
<style type="text/css">
  .box200 {
    width: 200px;
    height: 200px;
    border: 1px solid black;
    color: blue;
  }
</style>
...
This is before the box.
  <div class="box200">
This is a 200 by 200 pixel box with
  a 1px border.
  </div>
This is after the box.
...
```



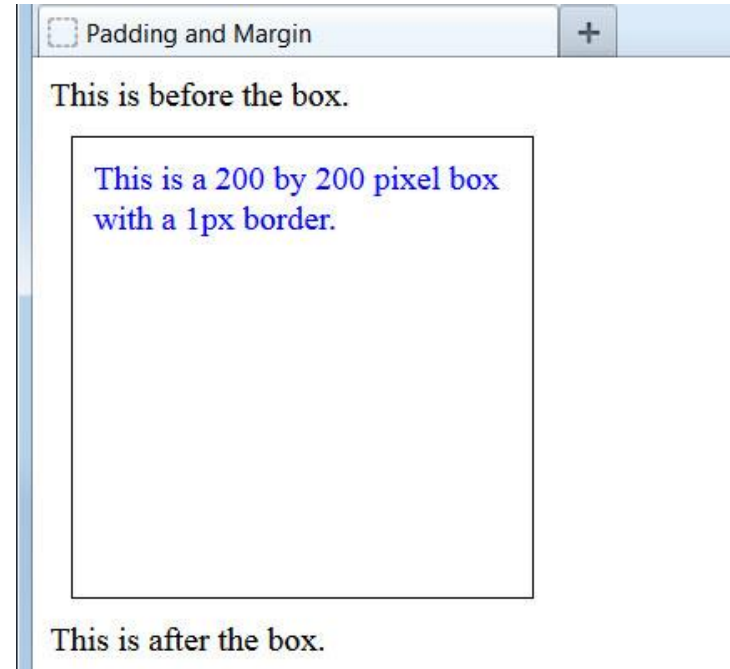
With the border set, we can see the exact space taken up on the page.

Notice that there is almost no space separating the text from the box border. Elements such as paragraphs, headers, and lists automatically insert padding and margins, but plain text does not do so.

Adding Padding and margin

Let's create some space between elements by adding both padding and margin:

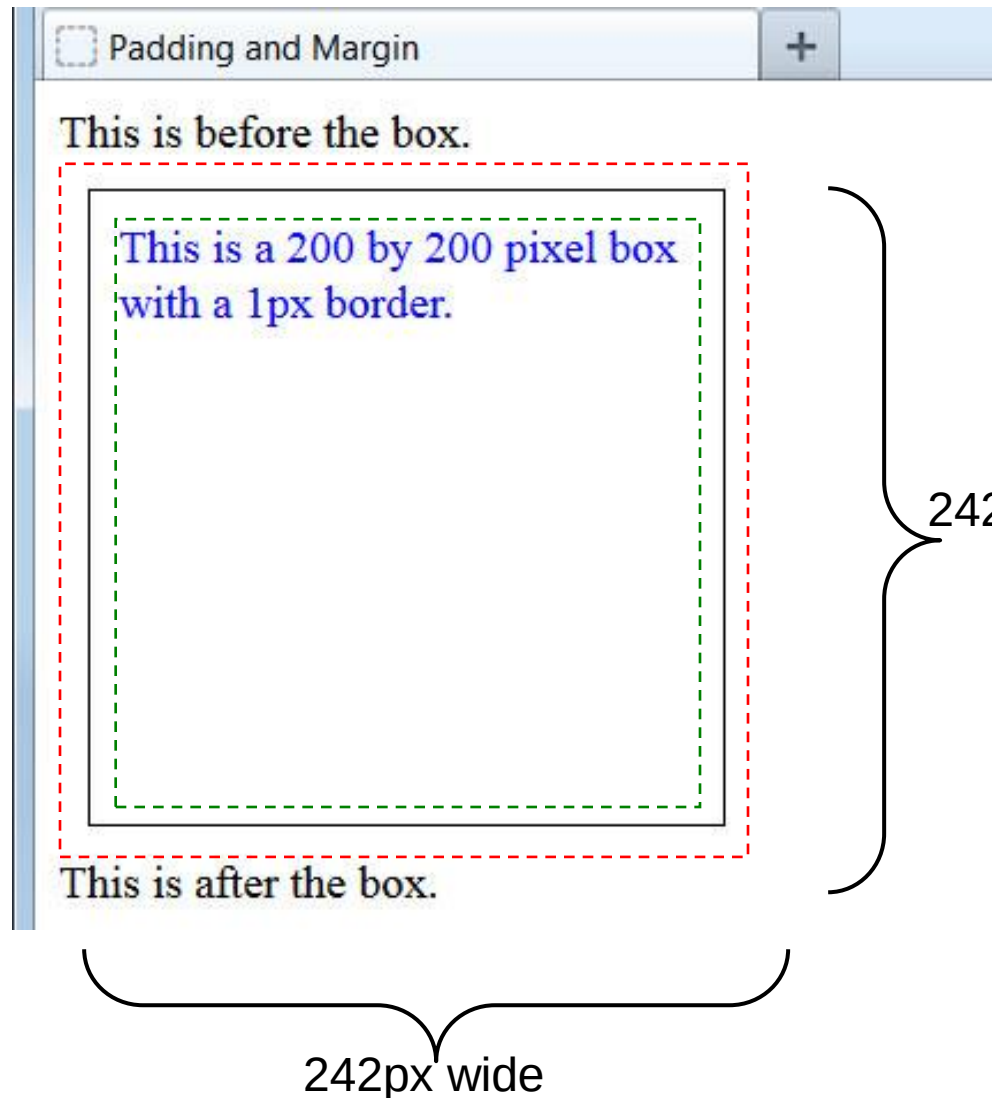
```
<style type="text/css">
  .box200 {
    width: 200px;
    height: 200px;
    border: 1px solid black;
    color: blue;
    padding: 10px;
    margin: 10px;
  }
</style>
...
```



The 10 pixel padding adds buffer space, on all four sides, between the content and border. The 10 pixel margin adds buffer space, on all four sides, between the border and surrounding elements.

Let's examine this a bit closer up to see exactly what is happening.

Padding and Margin Illustrated



The dotted red line shows the margin's outer boundary and the dotted green line shows the padding's inner boundary.

When we define the width and height of a `<div>` element, these dimensions apply only to the actual content, not to the padding, border, or margin.

Calculating Total Dimensions

When we are planning our page, we have to calculate exactly how much screen space a `<div>` or any other element will use. The formula is:

- Total element width = defined width + left padding + right padding + left border + right border + left margin + right margin.
- Total element height = defined height + top padding + bottom padding + top border + bottom border + top margin + bottom margin.

In our previous example:

- Total `<div>` width = 200px + 10px + 10px + 1px + 1px + 10px + 10px = 242px.
- Total `<div>` height = 200px + 10px + 10px + 1px + 1px + 10px + 10px = 242px.

Padding, borders, and margins do not have to be the same on all four sides, as they are in this example. Let's see how to set these individually.

Setting Individual Margins

We can set the specific margin sizes by using these four properties:

margin-top:

margin-right:

margin-bottom:

margin-left:

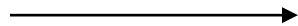
In practice, few web designers use these properties, preferring a **shorthand** method that condenses the declaration to a single line:

margin: 10px;



Sets all four margins to be 10px.

margin: 10px 5px;



Sets the top and bottom margins as 10px and the left and right margins as 5px.

margin: 20px 10px 5px 15px;



Sets the top margin as 20px, the right margin as 10px, the bottom margin as 5px, and the left margin as 15px.

Setting Individual Padding

We can set specific padding sizes by using these four properties:

padding-top:
padding-right:
padding-bottom:
padding-left:

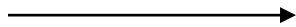
Again, these are rarely used. Instead, the same shorthand method is employed:

padding: 5px;



Sets padding as 5px on all four sides.

padding: 25px 5px;



Sets top and bottom padding as 25px and left and right padding as 5px.

padding: 50px 10px 15px 5px;



Sets top padding as 50px, right padding as 10px, bottom padding as 15px, and the left padding as 5px.

Setting Individual Borders

Customizing borders is more involved. Since they are visible on the page, we have to specify style, thickness, and color. Accordingly, there are more properties available:

<code>border-top-width:</code>	<code>border-top-style:</code>	<code>border-top-color:</code>
<code>border-right-width:</code>	<code>border-right-style:</code>	<code>border-right-color:</code>
<code>border-bottom-width:</code>	<code>border-bottom-style:</code>	<code>border-bottom-color:</code>
<code>border-left-width:</code>	<code>border-left-style:</code>	<code>border-left-color:</code>

To make things easier, we can use the shorthand method as before:

```
border-width: 10px;  
border-style: solid dashed;  
border-color: blue red orange gray;
```

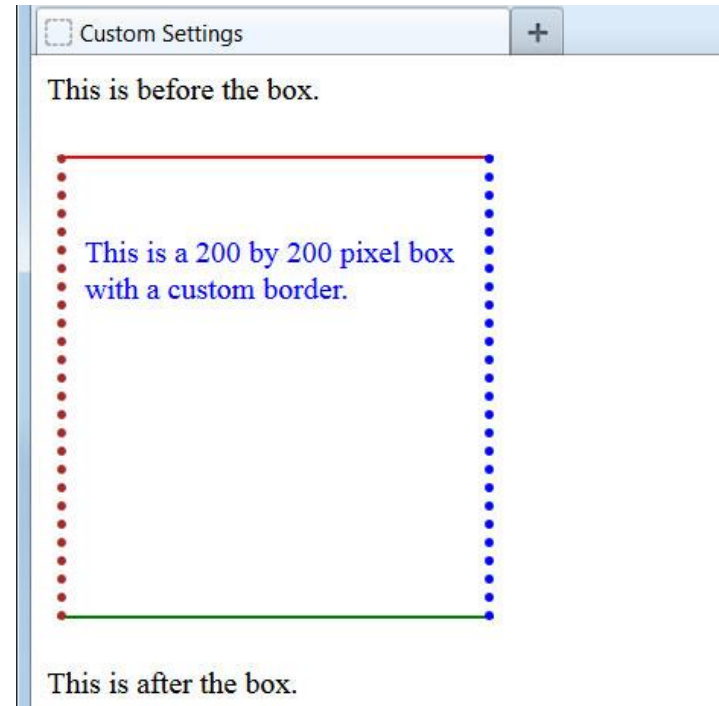
If the three border properties will be identical on all four sides, we can use a single-line border shorthand, as we did in an earlier lesson:

```
border: 5px solid blue;
```

Example of Customized Settings

Here we have set custom padding, borders, and margins:

```
<style type="text/css">
  .box200 {
    width: 200px;
    height: 200px;
    color: blue;
    padding: 40px 10px 0px 10px;
    margin: 25px 5px;
    border-width: 2px 5px;
    border-style: solid dotted;
    border-color: red blue green maroon;
  }
</style>
```



A helpful way to remember the order of shorthand settings is that it starts at noon and goes clockwise around: top, right, bottom, left.





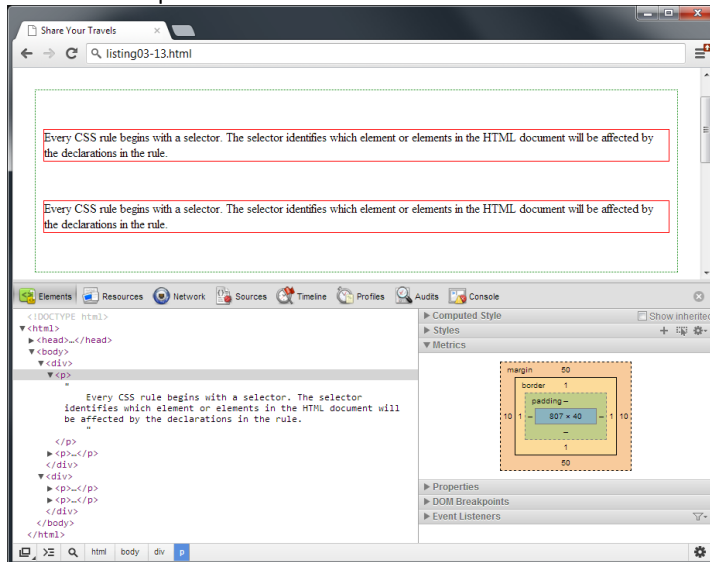
Developer tools

Developer Tools

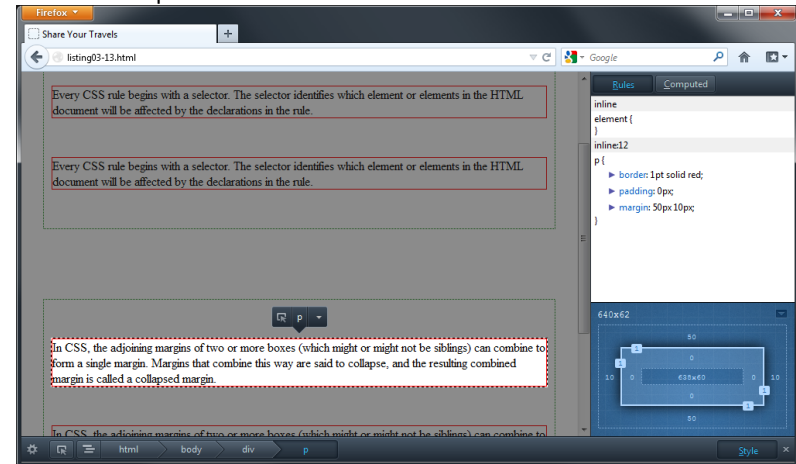
- Developer tools in current browsers make it significantly easier to examine and troubleshoot CSS than was the case a decade ago.
- You can use the various browsers' CSS inspection tools to examine, for instance, the box values for a selected element.

Developer Tools

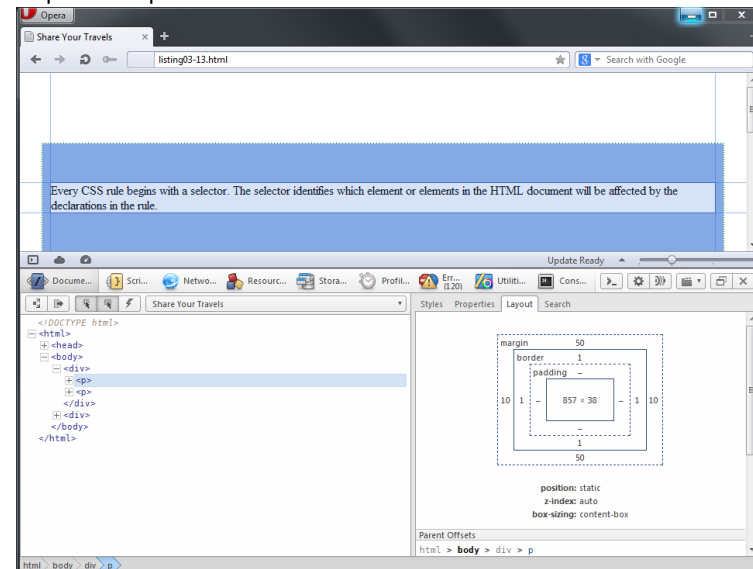
Chrome – Inspect Element



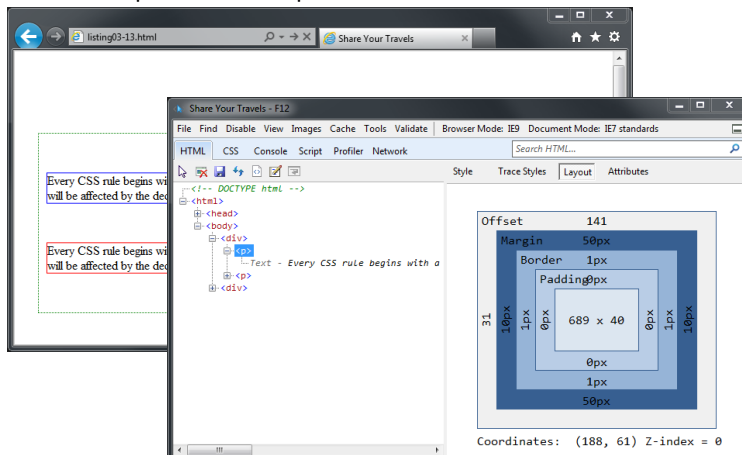
Firefox – Inspect



Opera – Inspect Element



Internet Explorer – Developer Tools



Cheat Sheet

- <http://htmlcheatsheet.com/css/>



CSS CheatSheet

How to use?



Properties

align-content	<i>behavior of the flex-wrap property</i>
align-items	<i>alignment for items inside the container</i>
align-self	<i>alignment for the selected item</i>
all	<i>changes all properties</i>
animation	<i>binds an animation to an element</i>
animation-delay	<i>delays animation start</i>
animation-direction	<i>reverse animation or in alternate cycles</i>
animation-duration	<i>animation duration in seconds or ms</i>
animation-fill-mode	<i>style when the animation is not playing</i>
animation-iteration-count	<i>number of an animation replays</i>
animation-name	<i>name for the @keyframes animation</i>

Basics

Syntax

```
selector{
  property: value;
  property2: value2;
}
```

Include external css file

```
<link rel="stylesheet" type="text/css" href="/style.css" />
```

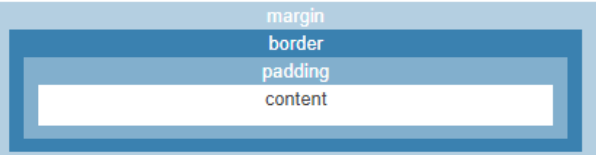
Internal styles

```
<style type="text/css">
  div { color: #444;}
</style>
```

Inline styles

```
<tag style="property: value"> </tag>
```

Box model



Color Picker

CSS generators

- <http://html-css-js.com/css/generator/>



COLOR **PICKER**

FONT **STYLER**

Grumpy wizards make
toxic brew for the
evil Queen and
Jack.

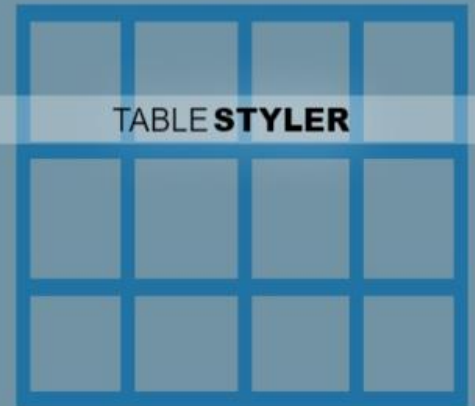


TABLE **STYLER**

Recommended Books & Sources of this presentation

FUNDAMENTALS of WEB DEVELOPMENT



RANDY CONNOLLY • RICARDO HOAR

HTML&CSS

design and build websites

JON DUCKETT



Eötvös Loránd University
Faculty of Informatics

**Thank you for your
attention**

Web-development I. course